



Universidade Estadual de Maringá

Centro de Tecnologia - CTC
Departamento de Informática - DIN
Ciência da Computação
Disciplina: Banco de Dados II

Eduarda Barragan da Silva - RA 139252
Julia Marques Sanches - RA 134678

Maringá, 2026

SUMÁRIO

1. Contexto do Exercício	3
1.1. Consultas Básicas	3
1.2. CRUD	6
2. Exercícios Práticos de NoSQL com MongoDB Atlas	8
2.1. Tema: StreamFlix — Plataforma de Streaming	8
2.2. Banco de dados	8
2.3. Criação da base de dados	8
2.3.1. Limpar a base antes de começar	8
2.3.2. Inserir usuários	9
2.3.3. Inserir conteúdos	9
2.3.4. Inserir assinaturas	10
2.3.5. Inserir avaliações	10
2.3.6. Inserir histórico de visualização	12
3. Exercícios evolutivos	12
3.1. Primeiros contatos com documentos e coleções	12
3.2. Operadores de comparação	22
3.3. Consultas com arrays	32
3.4. Objetos aninhados	42
3.5. Atualizações básicas	47
3.6. Atualizações com operadores	49
3.7. Operadores lógicos	52
3.8. Campos opcionais e flexibilidade NoSQL	59
3.9. Remoção de documentos	65

1. Contexto do Exercício

Uma pequena plataforma de streaming deseja organizar informações sobre filmes. Para isso, será utilizado um banco MongoDB chamado `streamflix`, com uma coleção chamada `filmes`.

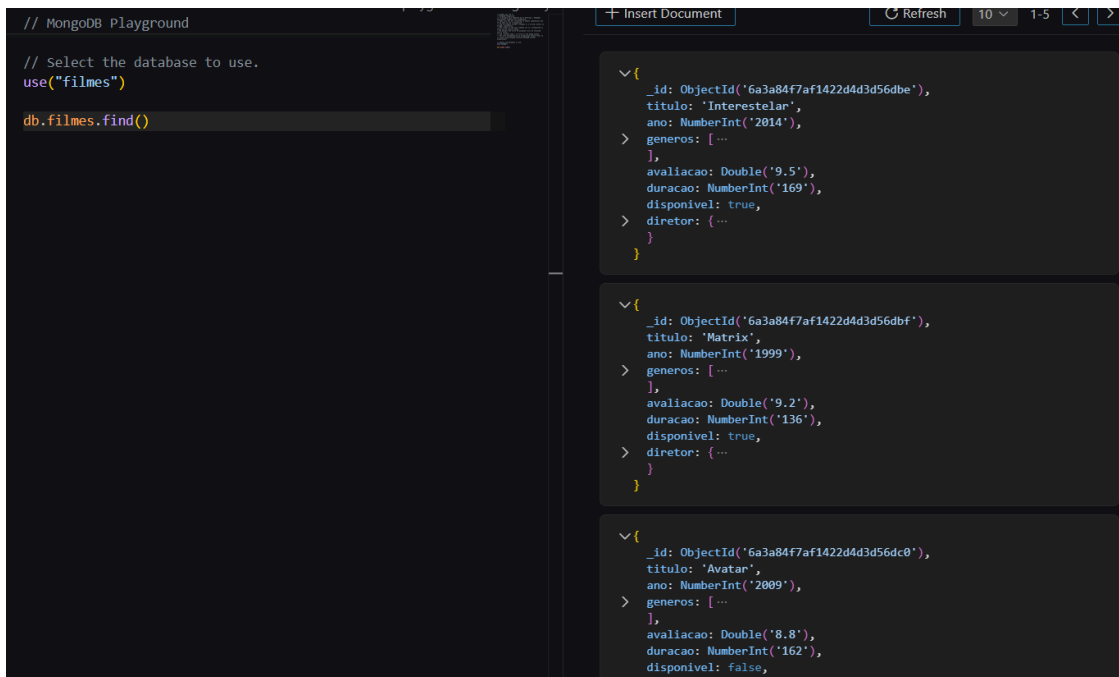
Cada filme será representado como um documento JSON.

1.1. Consultas Básicas

Exercício 1

Liste todos os filmes cadastrados.

R: `db.filmes.find()`



The screenshot shows the MongoDB Playground interface. On the left, the command `db.filmes.find()` is entered in the query editor. On the right, the results are displayed as a list of three JSON documents. Each document represents a movie with fields for `_id`, `titulo`, `ano`, `generos`, `avaliacao`, `duracao`, `disponivel`, and `diretor`.

```
// MongoDB Playground

// Select the database to use.
use("filmes")

db.filmes.find()
```

```
{
  "_id": ObjectId("6a3a84f7af1422d4d3d56dbe"),
  "titulo": "Interestelar",
  "ano": NumberInt("2014"),
  "generos": [ ... ],
  "avaliacao": Double("9.5"),
  "duracao": NumberInt("169"),
  "disponivel": true,
  "diretor": { ... }
}
```

```
{
  "_id": ObjectId("6a3a84f7af1422d4d3d56dbf"),
  "titulo": "Matrix",
  "ano": NumberInt("1999"),
  "generos": [ ... ],
  "avaliacao": Double("9.2"),
  "duracao": NumberInt("136"),
  "disponivel": true,
  "diretor": { ... }
}
```

```
{
  "_id": ObjectId("6a3a84f7af1422d4d3d56dc0"),
  "titulo": "Avatar",
  "ano": NumberInt("2009"),
  "generos": [ ... ],
  "avaliacao": Double("8.8"),
  "duracao": NumberInt("162"),
  "disponivel": false,
  "diretor": { ... }
}
```

```
},
}

{
  _id: ObjectId('6a3a84f7af1422d4d3d56dc1'),
  titulo: 'A Origem',
  ano: NumberInt('2010'),
  > generos: [...],
  avaliacao: NumberInt('9'),
  duracao: NumberInt('148'),
  disponivel: true,
  > diretor: {...}
}

{
  _id: ObjectId('6a3a84f7af1422d4d3d56dc2'),
  titulo: 'Divertida Mente',
  ano: NumberInt('2015'),
  > generos: [...],
  avaliacao: Double('8.7'),
  duracao: NumberInt('95'),
  disponivel: true,
  > diretor: {...}
}
```

Exercício 2

Busque apenas o filme Matrix.

R: `db.filmes.find({ titulo: "Matrix" })`

```
// Select the database to use.
use("filmes")

db.filmes.find({ titulo: "Matrix" })

{
  _id: ObjectId('6a3a84f7af1422d4d3d56dbf'),
  titulo: 'Matrix',
  ano: NumberInt('1999'),
  > generos: [...],
  avaliacao: Double('9.2'),
  duracao: NumberInt('136'),
  disponivel: true,
  > diretor: {...}
}
```

Exercício 3

Liste apenas os filmes disponíveis.

R: `db.filmes.find({ disponivel: true })`

```
// Select the database to use.
use("filmes")

db.filmes.find({ disponivel: true })
```

```
{
  "_id": ObjectId("6a3a84f7af1422d4d3d56dbe"),
  "titulo": "Interestelar",
  "ano": NumberInt("2014"),
  "generos": [ ... ],
  "avaliacao": Double("9.5"),
  "duracao": NumberInt("169"),
  "disponivel": true,
  "diretor": { ... }
}
```

```
{
  "_id": ObjectId("6a3a84f7af1422d4d3d56dbf"),
  "titulo": "Matrix",
  "ano": NumberInt("1999"),
  "generos": [ ... ],
  "avaliacao": Double("9.2"),
  "duracao": NumberInt("136"),
  "disponivel": true,
  "diretor": { ... }
}
```

```
{
  "_id": ObjectId("6a3a84f7af1422d4d3d56dc1"),
  "titulo": "A Origem",
  "ano": NumberInt("2010"),
  "generos": [ ... ],
  "avaliacao": NumberInt("9"),
  "duracao": NumberInt("148"),
  "disponivel": true,
  "diretor": { ... }
}
```

```
{
  "_id": ObjectId("6a3a84f7af1422d4d3d56dc2"),
  "titulo": "Divertida Mente",
  "ano": NumberInt("2015"),
  "generos": [ ... ],
  "avaliacao": Double("8.7"),
  "duracao": NumberInt("95"),
  "disponivel": true,
  "diretor": { ... }
}
```

Exercício 4

Busque o filme cujo título seja Avatar.

R: `db.filmes.findOne({ titulo: "Avatar" })`

```
ers > eduar > OneDrive > Documentos > UEM > TERCEIRO > BDII > JS playground-1.mongodb.js > 1 {
playgrounds/ 2 {
11 // Select the database to use. 3   _id: ObjectId('6a3a84f7af1422d4d3d56dc0'),
12 use("filmes") 4   titulo: 'Avatar',
13 14 5   ano: NumberInt('2009'),
14 db.filmes.findOne({ titulo: "Avatar" }) 6   generos: [
15 16 7     'Aventura',
16 17 8     'Ficção Científica'
17 18 9   ],
18 10   avaliacao: Double('8.8'),
11   duracao: NumberInt('162'),
12   disponivel: false,
13   diretor: {
14     nome: 'James Cameron',
15     pais: 'Canadá'
16   }
17 }
18 }
```

1.2. CRUD

O CRUD representa as quatro operações básicas sobre dados:

- Create: criar/insserir;
- Read: ler/consultar;
- Update: atualizar;
- Delete: remover.

Exercício 5 — Create

Insira um novo filme.

```
R: db.filmes.insertOne({
  titulo: "O Senhor dos Anéis",
  ano: 2001,
  generos: ["Fantasia", "Aventura"],
  avaliacao: 9.1,
  duracao: 178,
  disponivel: true,
  diretor: {
    nome: "Peter Jackson",
    pais: "Nova Zelândia"
  }
})
```



The screenshot shows a MongoDB playground interface. On the left, the command window displays the following code:

```
> eduar > OneDrive > Documentos > UEM > TERCEIRO > BDII > JS playground-1.mongodb.js >
// https://www.mongodb.com/docs/mongodb-vscode/playgrounds/

// Select the database to use.
use("filmes")

db.filmes.insertOne({
  titulo: "O Senhor dos Anéis",
  ano: 2001,
  generos: ["Fantasia", "Aventura"],
  avaliacao: 9.1,
  duracao: 178,
  disponivel: true,
  diretor: {
    nome: "Peter Jackson",
    pais: "Nova Zelândia"
  }
})
```

On the right, the output window shows the following JSON response:

```
1 {
2   acknowledged: true,
3   insertedId: ObjectId('6a3a8720d68dcb3f395f0be')
4 }
```

Exercício 6 — Read

Consulte o filme inserido.

```
R: db.filmes.find({ titulo: "O Senhor dos Anéis" })
```

```
// Select the database to use.
use("filmes")

db.filmes.find({ titulo: "O Senhor dos Anéis" })
```

```
{
  "_id": ObjectId('6a3a8720d68dcb3f395f0be'),
  "titulo": "O Senhor dos Anéis",
  "ano": NumberInt('2001'),
  "generos": [ ... ],
  "avaliacao": Double('9.1'),
  "duracao": NumberInt('178'),
  "disponivel": true,
  "diretor": { ... }
}
```

Exercício 7 — Update

Atualize a avaliação de Avatar para 9.0.

```
R: db.filmes.updateOne(
  { titulo: "Avatar" },
  { $set: { avaliacao: 9.0 } }
)
```

```
ers > eduar > OneDrive > Documentos > UEM > TERCEIRO > BDII > JS playground-1.mongodb.js > 1 {
12 // Select the database to use. 2 {
13 use("filmes") 3 acknowledged: true,
14 4 insertedId: null,
15 db.filmes.updateOne( 5 matchedCount: NumberInt('1'),
16 { titulo: "Avatar" }, 6 modifiedCount: NumberInt('1'),
17 { $set: { avaliacao: 9.0 } } 7 upsertedCount: NumberInt('0')
18 ) }
```

Exercício 8 — Delete

Remova o filme O Senhor dos Anéis.

```
R: db.filmes.deleteOne({ titulo: "O Senhor dos Anéis" })
```

```
> eduar > OneDrive > Documentos > UEM > TERCEIRO > BDII > JS playground-1.mongodb.js > 1 {
playgrounds/ 2 {
3 acknowledged: true,
4 deletedCount: NumberInt('1')
}
1 // Select the database to use.
2 use("filmes")
3
4
5 db.filmes.deleteOne({ titulo: "O Senhor dos Anéis" })
```

2. Exercícios Práticos de NoSQL com MongoDB Atlas

2.1. Tema: StreamFlix — Plataforma de Streaming

Este material apresenta uma base de dados maior para prática com MongoDB e uma lista de exercícios em níveis evolutivos. A proposta é utilizar o MongoDB Atlas, preferencialmente com o VS Code e a extensão MongoDB for VS Code, executando os comandos em arquivos .mongodb.

2.2. Banco de dados

Nome do banco:

```
use("streamflix")
```

Coleções utilizadas:

- usuarios
- conteudos
- avaliacoes
- assinaturas
- Historico

2.3. Criação da base de dados

Execute os comandos abaixo em um Playground MongoDB no VS Code.

2.3.1. Limpar a base antes de começar

```
use("streamflix")
```

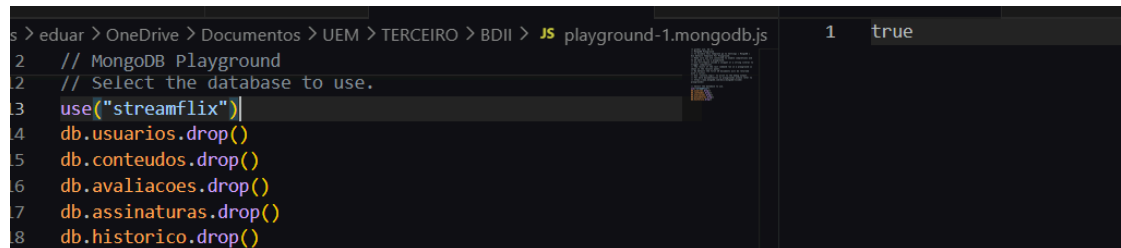
```
db.usuarios.drop()
```

```
db.conteudos.drop()
```

```
db.avaliacoes.drop()
```

```
db.assinaturas.drop()
```

```
db.historico.drop()
```



The screenshot shows a VS Code terminal window with a file explorer on the left and a terminal on the right. The file explorer shows the path: s > eduar > OneDrive > Documentos > UEM > TERCEIRO > BDII > JS playground-1.mongodb.js. The terminal window has a title bar that says "1 true". The terminal content is as follows:

```
s > eduar > OneDrive > Documentos > UEM > TERCEIRO > BDII > JS playground-1.mongodb.js
2 // MongoDB Playground
2 // Select the database to use.
3 use("streamflix")
4 db.usuarios.drop()
5 db.conteudos.drop()
6 db.avaliacoes.drop()
7 db.assinaturas.drop()
8 db.historico.drop()
```


2.3.2. Inserir usuários

[illegible]

2.3.3. Inserir conteúdos

A coleção `conteudos` armazena filmes, séries e documentários. Essa decisão mostra uma característica comum em bancos NoSQL: documentos com estruturas semelhantes, mas não necessariamente idênticas, podem ser armazenados na mesma coleção.

```
> educar > OneDrive > Documentos > UEM > TERCEIRO > BDII > JS playground-1.mongodb.js >

use("streamflix")

db.conteudos.insertMany([
  {
    titulo: "Interestelar",
    tipo: "filme",
    ano: 2014,
    generos: ["Drama", "Ficção Científica"],
    avaliacaoMedia: 9.5,
    duracaoMinutos: 169,
    disponivel: true,
    visualizacoes: 2500000,
    diretor: {
      nome: "Christopher Nolan",
      pais: "Reino Unido"
    },
    elenco: ["Matthew McConaughey", "Anne Hathaway",
    "Jessica Chastain"]
  },
  {
    titulo: "Matrix",
    tipo: "filme",
    ano: 1999,
    generos: ["Ação", "Ficção Científica"],
    avaliacaoMedia: 9.2,
    duracaoMinutos: 136,
    disponivel: true,
    acknowledged: true,
    insertedIds: {
      '0': ObjectId('6a3a898707a0ceaf28f2ff0'),
      '1': ObjectId('6a3a898707a0ceaf28f2ff1'),
      '2': ObjectId('6a3a898707a0ceaf28f2ff2'),
      '3': ObjectId('6a3a898707a0ceaf28f2ff3'),
      '4': ObjectId('6a3a898707a0ceaf28f2ff4'),
      '5': ObjectId('6a3a898707a0ceaf28f2ff5'),
      '6': ObjectId('6a3a898707a0ceaf28f2ff6'),
      '7': ObjectId('6a3a898707a0ceaf28f2ff7'),
      '8': ObjectId('6a3a898707a0ceaf28f2ff8'),
      '9': ObjectId('6a3a898707a0ceaf28f2ff9'),
      '10': ObjectId('6a3a898707a0ceaf28f2ffa'),
      '11': ObjectId('6a3a898707a0ceaf28f2ffb'),
      '12': ObjectId('6a3a898707a0ceaf28f2ffc'),
      '13': ObjectId('6a3a898707a0ceaf28f2ffd'),
      '14': ObjectId('6a3a898707a0ceaf28f2ffe')
    }
  }
])
```

2.3.4. Inserir assinaturas

```
MongoDB for VS Code  MongoDB JS playground-1.mongodb.js  Playground Result  Playground Result: times.times

s > eduar > OneDrive > Documentos > UEM > TERCEIRO > BDII > JS playground-1.mongodb.js > 1 {
12 // Select the database to use. 2 acknowledged: true,
13 use("streamflix") 3 insertedIds: {
14 4 '0': ObjectId('6a3a89da4b933bffa6f20cd5'),
15 db.assinaturas.insertMany([ 5 '1': ObjectId('6a3a89da4b933bffa6f20cd6'),
16 { 6 '2': ObjectId('6a3a89da4b933bffa6f20cd7'),
17 { 7 '3': ObjectId('6a3a89da4b933bffa6f20cd8'),
18 usuarioEmail: "ana@email.com", 8 '4': ObjectId('6a3a89da4b933bffa6f20cd9'),
19 plano: "Premium", 9 '5': ObjectId('6a3a89da4b933bffa6f20cda')
20 valorMensal: 49.90, 10 }
21 ativo: true, 11 }
22 formaPagamento: "Cartão de crédito",
23 beneficios: ["4 telas", "4K", "download"]
24 },
25 {
26 usuarioEmail: "carlos@email.com",
27 plano: "Básico",
28 valorMensal: 24.90,
29 ativo: false,
30 formaPagamento: "Boleto",
31 beneficios: ["1 tela", "HD"]
32 },
33 {
34 usuarioEmail: "fernanda@email.com",
35 plano: "Padrão",
valorMensal: 34.90,
}
```

2.3.5. Inserir avaliações

```
MongoDB for VS Code  MongoDB JS playground-1.mongodb.js  Playground Result  Playground Result: times.times

ers > eduar > OneDrive > Documentos > UEM > TERCEIRO > BDII > JS playground-1.mongodb.js > 1 {
12 // Select the database to use. 2 acknowledged: true,
13 use("streamflix") 3 insertedIds: {
14 4 '0': ObjectId('6a3a8a072adb642e1f54b4eb')
15 db.avaliacoes.insertMany([ 5 '1': ObjectId('6a3a8a072adb642e1f54b4ec')
16 { 6 '2': ObjectId('6a3a8a072adb642e1f54b4ed')
17 { 7 '3': ObjectId('6a3a8a072adb642e1f54b4ee')
18 usuarioEmail: "ana@email.com", 8 '4': ObjectId('6a3a8a072adb642e1f54b4ef')
19 tituloConteudo: "Interestelar", 9 '5': ObjectId('6a3a8a072adb642e1f54b4f0')
20 nota: 10, 10 '6': ObjectId('6a3a8a072adb642e1f54b4f1')
21 comentario: "Excelente filme, muito emocionante.", 11 '7': ObjectId('6a3a8a072adb642e1f54b4f2')
22 data: ISODate("2026-03-10") 12 }
23 }, 13 }
24 {
25 usuarioEmail: "carlos@email.com",
26 tituloConteudo: "Matrix",
27 nota: 9,
28 comentario: "Um clássico da ficção científica.",
29 data: ISODate("2026-03-11")
30 },
31 {
32 usuarioEmail: "fernanda@email.com",
33 tituloConteudo: "Dark",
34 nota: 9,
35 comentario: "Série complexa e envolvente.",
36 data: ISODate("2026-03-12")
37 },
38 {
39 usuarioEmail: "marina@email.com",
40 tituloConteudo: "Planeta Terra",
41 nota: 10,
42 comentario: "Documentário visualmente impressionante.",
43 data: ISODate("2026-03-13")
}
```

2.3.6. Inserir histórico de visualização

```
rs > eduar > OneDrive > Documentos > UEM > TERCEIRO > BDII > JS playground-1.mongodb.js >
12 // Select the database to use.
13 use("streamflix")
14
15 db.historico.insertMany([
16   {
17     usuarioEmail: "ana@email.com",
18     tituloConteudo: "Interestelar",
19     tipo: "filme",
20     progressoPercentual: 100,
21     finalizado: true,
22     dataVisualizacao: ISODate("2026-04-01")
23   },
24   {
25     usuarioEmail: "ana@email.com",
26     tituloConteudo: "Dark",
27     tipo: "serie",
28     temporada: 1,
29     episodio: 3,
30     progressoPercentual: 45,
31     finalizado: false,
32     dataVisualizacao: ISODate("2026-04-03")
33   },
34   {
35     usuarioEmail: "carlos@email.com",
36     tituloConteudo: "Matrix",
37     tipo: "filme",
38     progressoPercentual: 100,
39     finalizado: true,
40     dataVisualizacao: ISODate("2026-04-02")
41   },
42   {
43     usuarioEmail: "fernanda@email.com",
44     tituloConteudo: "Matrix",
45     tipo: "filme",
46     progressoPercentual: 100,
47     finalizado: true,
48     dataVisualizacao: ISODate("2026-04-02")
49   }
50 ])
51
52 {
53   acknowledged: true,
54   insertedIds: {
55     '0': ObjectId('6a3a8a53f8a30aa20dba62a8'),
56     '1': ObjectId('6a3a8a53f8a30aa20dba62a9'),
57     '2': ObjectId('6a3a8a53f8a30aa20dba62aa'),
58     '3': ObjectId('6a3a8a53f8a30aa20dba62ab'),
59     '4': ObjectId('6a3a8a53f8a30aa20dba62ac'),
60     '5': ObjectId('6a3a8a53f8a30aa20dba62ad'),
61     '6': ObjectId('6a3a8a53f8a30aa20dba62ae'),
62     '7': ObjectId('6a3a8a53f8a30aa20dba62af'),
63     '8': ObjectId('6a3a8a53f8a30aa20dba62b0')
64   }
65 }
```

3. Exercícios evolutivos

3.1. Primeiros contatos com documentos e coleções

Exercício 1

Liste todos os documentos da coleção `usuarios`.

R: `db.usuarios.find()`

VS Code interface showing MongoDB Playground results for streamflix.usuarios.

Left Panel (Code Editor):

```
// MongoDB Playground
playgrounds/

// Select the database to use.
use("streamflix")
db.usuarios.find()
```

Right Panel (Results):

Playground Result: streamflix.usuarios

10 1-10

Insert Document Refresh

Results (3 documents):

- ```
{
 _id: ObjectId('6a3a8911551fbf74bf9873de'),
 nome: 'Ana Souza',
 email: 'ana@email.com',
 idade: NumberInt('22'),
 cidade: 'Curitiba',
 estado: 'PR',
 interesses: [...],
 ativo: true,
 endereco: { ... }
}
```
- ```
{
  _id: ObjectId('6a3a8911551fbf74bf9873df'),
  nome: 'Carlos Lima',
  email: 'carlos@email.com',
  idade: NumberInt('31'),
  cidade: 'Maringá',
  estado: 'PR',
  interesses: [ ... ],
  ativo: false,
  endereco: { ... }
}
```
- ```
{
 _id: ObjectId('6a3a8911551fbf74bf9873e0'),
 nome: 'Fernanda Rocha',
 email: 'fernanda@email.com',
 idade: NumberInt('27'),
 cidade: 'Londrina',
 estado: 'PR',
 interesses: [...],
 ativo: true
}
```

VS Code interface showing MongoDB Playground results for streamflix.usuarios.

Left Panel (Code Editor):

```
// MongoDB Playground
playgrounds/

// Select the database to use.
use("streamflix")
db.usuarios.find()
```

Right Panel (Results):

Playground Result: streamflix.usuarios

10 1-10

Insert Document Refresh

Results (3 documents):

- ```
{
  _id: ObjectId('6a3a8911551fbf74bf9873e1'),
  nome: 'João Mendes',
  email: 'joao@email.com',
  idade: NumberInt('19'),
  cidade: 'Curitiba',
  estado: 'PR',
  interesses: [ ... ],
  ativo: true
}
```
- ```
{
 _id: ObjectId('6a3a8911551fbf74bf9873e2'),
 nome: 'Marina Costa',
 email: 'marina@email.com',
 idade: NumberInt('35'),
 cidade: 'São Paulo',
 estado: 'SP',
 interesses: [...],
 ativo: true,
 premium: true
}
```
- ```
{
  _id: ObjectId('6a3a8911551fbf74bf9873e3'),
  nome: 'Rafael Oliveira',
  email: 'rafael@email.com',
  idade: NumberInt('42'),
  cidade: 'Rio de Janeiro',
  estado: 'RJ',
  interesses: [ ... ],
  ativo: false
}
```

The screenshot shows the MongoDB Playground interface in VS Code. The left pane contains the following code:

```
2 // MongoDB Playground
11 playgrounds/
12 // Select the database to use.
13 use("streamflix")
14 db.usuarios.find()
```

The right pane displays the results of the query, showing four documents from the `streamflix.usuarios` collection. Each document is a JSON object with fields: `_id`, `nome`, `email`, `idade`, `cidade`, `estado`, `interesses`, `ativo`, and `telefone` (or `premium` for the last document).

```
{
  "_id": ObjectId("6a3a8911551fbf74bf9873e4"),
  "nome": "Beatriz Nunes",
  "email": "beatriz@email.com",
  "idade": NumberInt("24"),
  "cidade": "Maringá",
  "estado": "PR",
  "interesses": [ ... ],
  "ativo": true,
  "telefone": "4499999-0000"
}
```

```
{
  "_id": ObjectId("6a3a8911551fbf74bf9873e5"),
  "nome": "Lucas Ferreira",
  "email": "lucas@email.com",
  "idade": NumberInt("29"),
  "cidade": "Florianópolis",
  "estado": "SC",
  "interesses": [ ... ],
  "ativo": true
}
```

```
{
  "_id": ObjectId("6a3a8911551fbf74bf9873e6"),
  "nome": "Patrícia Alves",
  "email": "patricia@email.com",
  "idade": NumberInt("38"),
  "cidade": "Belo Horizonte",
  "estado": "MG",
  "interesses": [ ... ],
  "ativo": true
}
```

```
{
  "_id": ObjectId("6a3a8911551fbf74bf9873e7"),
  "nome": "Eduardo Martins",
  "email": "eduardo@email.com",
  "idade": NumberInt("33"),
  "cidade": "Curitiba",
  "estado": "PR",
  "interesses": [ ... ],
  "ativo": true,
  "premium": false
}
```

Exercício 2

Liste todos os documentos da coleção `conteudos`.

R: `db.conteudos.find()`

ers > eduar > OneDrive > Documentos > UEM > TERCEIRO > BDII > JS playground-1.mongodb.js

```
2 // MongoDB Playground
  playgrounds/
11
12 // Select the database to use.
13 use("streamflix")
14 db.conteudos.find()
15
```

+ Insert Document Refresh 10 1-10 < >

```
{
  _id: ObjectId('6a3a898707a0ceaf28f2ff0'),
  titulo: 'Interestelar',
  tipo: 'filme',
  ano: NumberInt('2014'),
  generos: [...],
  avaliacaoMedia: Double('9.5'),
  duracaoMinutos: NumberInt('169'),
  disponivel: true,
  visualizacoes: NumberInt('250000'),
  diretor: {...},
  elenco: [...],
}
```

```
{
  _id: ObjectId('6a3a898707a0ceaf28f2ff1'),
  titulo: 'Matrix',
  tipo: 'filme',
  ano: NumberInt('1999'),
  generos: [...],
  avaliacaoMedia: Double('9.2'),
  duracaoMinutos: NumberInt('136'),
  disponivel: true,
  visualizacoes: NumberInt('310000'),
  diretor: {...},
  elenco: [...],
}
```

```
{
  _id: ObjectId('6a3a898707a0ceaf28f2ff2'),
  titulo: 'Avatar',
  tipo: 'filme',
  ano: NumberInt('2009'),
  generos: [...],
  avaliacaoMedia: Double('8.8'),
  duracaoMinutos: NumberInt('162'),
  disponivel: true,
  visualizacoes: NumberInt('280000'),
  diretor: {...},
  premios: [...],
}
```

ers > eduar > OneDrive > Documentos > UEM > TERCEIRO > BDII > JS playground-1.mongodb.js

```
2 // MongoDB Playground
  playgrounds/
11
12 // Select the database to use.
13 use("streamflix")
14 db.conteudos.find()
15
```

+ Insert Document Refresh 10 1-10 < >

```
{
  _id: ObjectId('6a3a898707a0ceaf28f2ff2'),
  titulo: 'Avatar',
  tipo: 'filme',
  ano: NumberInt('2009'),
  generos: [...],
  avaliacaoMedia: Double('8.8'),
  duracaoMinutos: NumberInt('162'),
  disponivel: true,
  visualizacoes: NumberInt('280000'),
  diretor: {...},
  premios: [...],
}
```

```
{
  _id: ObjectId('6a3a898707a0ceaf28f2ff3'),
  titulo: 'O Senhor dos Anéis: A Sociedade do Anel',
  tipo: 'filme',
  ano: NumberInt('2001'),
  generos: [...],
  avaliacaoMedia: Double('9.4'),
  duracaoMinutos: NumberInt('178'),
  disponivel: true,
  visualizacoes: NumberInt('220000'),
  diretor: {...},
  premios: [...],
}
```

```
{
  _id: ObjectId('6a3a898707a0ceaf28f2ff4'),
  titulo: 'A Origem',
  tipo: 'filme',
  ano: NumberInt('2010'),
  generos: [...],
  avaliacaoMedia: Double('9.1'),
}
```

Extension: MongoDB for VS Code | MongoDB | JS playground-1.mongodb.js | round Result: streamflix.usuarios | Playground Result: streamflix.conteudos

```
2 // MongoDB Playground
  playgrounds/
11
12 // Select the database to use.
13 use("streamflix")
14 db.conteudos.find()
15
```

+ Insert Document

Refresh 10 1-10

```
{
  tipo: 'filme',
  ano: NumberInt('2010'),
  generos: [ ... ],
  avaliacaoMedia: Double('9.1'),
  duracaoMinutos: NumberInt('148'),
  disponivel: true,
  visualizacoes: NumberInt('1950000'),
  diretor: { ... }
}
```

```
{
  _id: ObjectId('6a3a898707a0ceaf28f2ff5'),
  titulo: 'Divertida Mente',
  tipo: 'filme',
  ano: NumberInt('2015'),
  generos: [ ... ],
  avaliacaoMedia: Double('8.7'),
  duracaoMinutos: NumberInt('95'),
  disponivel: true,
  visualizacoes: NumberInt('1750000')
}
```

```
{
  _id: ObjectId('6a3a898707a0ceaf28f2ff6'),
  titulo: 'Cidade de Deus',
  tipo: 'filme',
  ano: NumberInt('2002'),
  generos: [ ... ],
  avaliacaoMedia: NumberInt('9'),
  duracaoMinutos: NumberInt('130'),
  disponivel: false,
  visualizacoes: NumberInt('980000'),
  diretor: { ... }
}
```

Extension: MongoDB for VS Code | MongoDB | JS playground-1.mongodb.js | round Result: streamflix.usuarios | Playground Result: streamflix.conteudos

```
2 // MongoDB Playground
  playgrounds/
11
12 // Select the database to use.
13 use("streamflix")
14 db.conteudos.find()
15
```

+ Insert Document

Refresh 10 1-10

```
{
  _id: ObjectId('6a3a898707a0ceaf28f2ff7'),
  titulo: 'Dark',
  tipo: 'serie',
  ano: NumberInt('2017'),
  generos: [ ... ],
  avaliacaoMedia: Double('9.1'),
  temporadas: NumberInt('3'),
  episodios: NumberInt('26'),
  disponivel: true,
  visualizacoes: NumberInt('2100000'),
  classificacao: '16+'
}
```

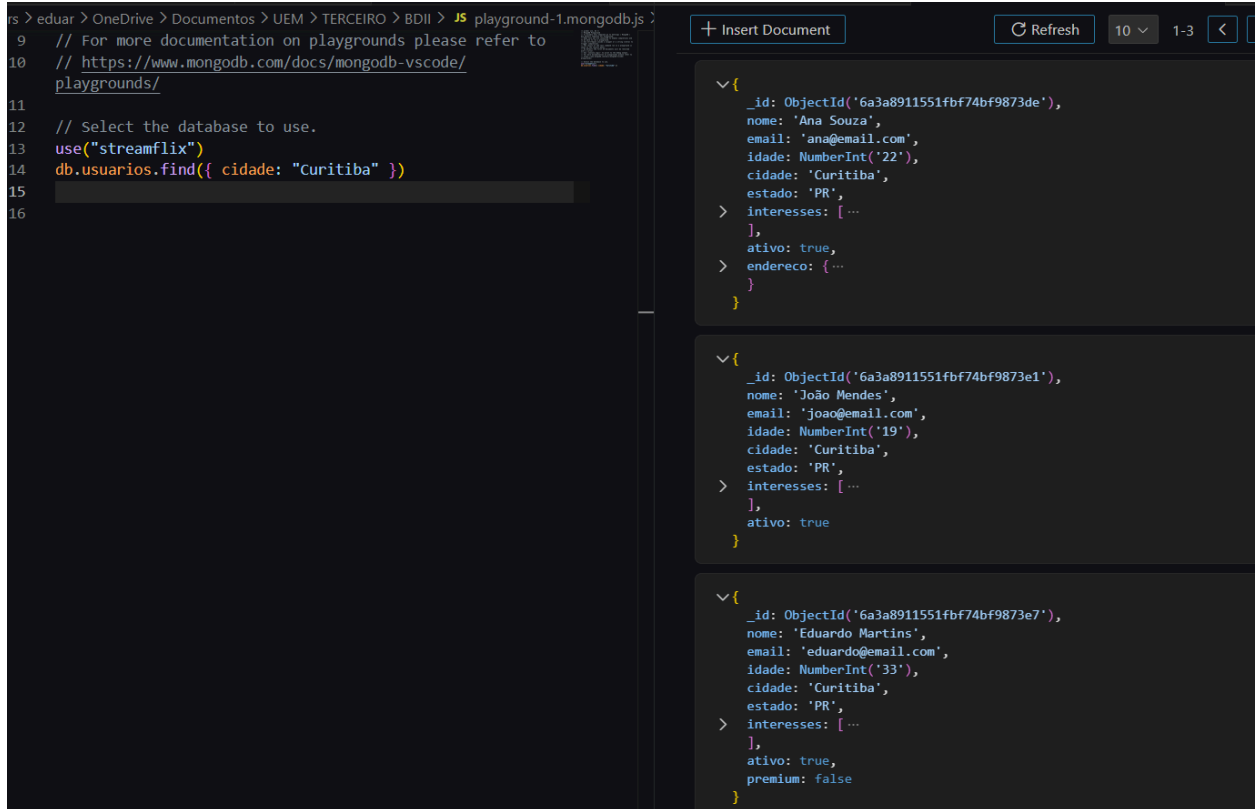
```
{
  _id: ObjectId('6a3a898707a0ceaf28f2ff8'),
  titulo: 'Breaking Bad',
  tipo: 'serie',
  ano: NumberInt('2008'),
  generos: [ ... ],
  avaliacaoMedia: Double('9.8'),
  temporadas: NumberInt('5'),
  episodios: NumberInt('62'),
  disponivel: true,
  visualizacoes: NumberInt('4200000'),
  classificacao: '18+'
}
```

```
{
  _id: ObjectId('6a3a898707a0ceaf28f2ff9'),
  titulo: 'Stranger Things',
  tipo: 'serie',
  ano: NumberInt('2016'),
  generos: [ ... ],
  avaliacaoMedia: Double('8.9'),
  temporadas: NumberInt('4'),
  episodios: NumberInt('34'),
  disponivel: true
}
```

Exercício 3

Liste todos os usuários da cidade de Curitiba.

R: `db.usuarios.find({ cidade: "Curitiba" })`



The screenshot shows a VS Code editor with a MongoDB playground script on the left and the results of the query on the right. The script on the left includes comments about documentation and a link to the MongoDB VS Code playgrounds, followed by the command `use("streamflix")` and the query `db.usuarios.find({ cidade: "Curitiba" })`. The results on the right show three documents from the `usuarios` collection, all with `cidade: "Curitiba"`. Each document contains fields for `_id`, `nome`, `email`, `idade`, `estado`, `interesses`, `ativo`, and `endereco`. The first document is for Ana Souza (age 22), the second for João Mendes (age 19), and the third for Eduardo Martins (age 33). The third document also includes a `premium: false` field.

```
rs > eduar > OneDrive > Documentos > UEM > TERCEIRO > BDII > JS playground-1.mongodb.js
9 // For more documentation on playgrounds please refer to
10 // https://www.mongodb.com/docs/mongodb-vscode/
11 // playgrounds/
12 // Select the database to use.
13 use("streamflix")
14 db.usuarios.find({ cidade: "Curitiba" })
15
16
```

Results:

```
{
  "_id": ObjectId('6a3a8911551fbf74bf9873de'),
  "nome": "Ana Souza",
  "email": "ana@email.com",
  "idade": NumberInt('22'),
  "cidade": "Curitiba",
  "estado": "PR",
  "interesses": [...],
  "ativo": true,
  "endereco": {...}
}
```

```
{
  "_id": ObjectId('6a3a8911551fbf74bf9873e1'),
  "nome": "João Mendes",
  "email": "joao@email.com",
  "idade": NumberInt('19'),
  "cidade": "Curitiba",
  "estado": "PR",
  "interesses": [...],
  "ativo": true
}
```

```
{
  "_id": ObjectId('6a3a8911551fbf74bf9873e7'),
  "nome": "Eduardo Martins",
  "email": "eduardo@email.com",
  "idade": NumberInt('33'),
  "cidade": "Curitiba",
  "estado": "PR",
  "interesses": [...],
  "ativo": true,
  "premium": false
}
```

Exercício 4

Liste todos os conteúdos do tipo filme.

R: `db.conteudos.find({ tipo: "filme" })`

rs > eduar > OneDrive > Documentos > UEM > TERCEIRO > BDII > JS playground-1.mongodb.js

```
9 // For more documentation on playgrounds please refer to
10 // https://www.mongodb.com/docs/mongodb-vscode/
11 // playgrounds/
12 // Select the database to use.
13 use("streamflix")
14 db.conteudos.find({ tipo: "filme" })
15
16
17
```

+ Insert Document Refresh 10 1-8

```
{
  _id: ObjectId('6a3a898707a0ceaf28f2ff0'),
  titulo: 'Interestelar',
  tipo: 'filme',
  ano: NumberInt('2014'),
  generos: [ ... ],
  avaliacaoMedia: Double('9.5'),
  duracaoMinutos: NumberInt('169'),
  disponivel: true,
  visualizacoes: NumberInt('250000'),
  diretor: { ... },
  elenco: [ ... ]
}
```

```
{
  _id: ObjectId('6a3a898707a0ceaf28f2ff1'),
  titulo: 'Matrix',
  tipo: 'filme',
  ano: NumberInt('1999'),
  generos: [ ... ],
  avaliacaoMedia: Double('9.2'),
  duracaoMinutos: NumberInt('136'),
  disponivel: true,
  visualizacoes: NumberInt('310000'),
  diretor: { ... },
  elenco: [ ... ]
}
```

```
{
  _id: ObjectId('6a3a898707a0ceaf28f2ff2'),
  titulo: 'Avatar',
  tipo: 'filme',
  ano: NumberInt('2009'),
  generos: [ ... ],
  avaliacaoMedia: Double('8.8'),
  duracaoMinutos: NumberInt('162'),
  disponivel: true,
  visualizacoes: NumberInt('280000'),
  diretor: { ... },
  premios: [ ... ]
}
```

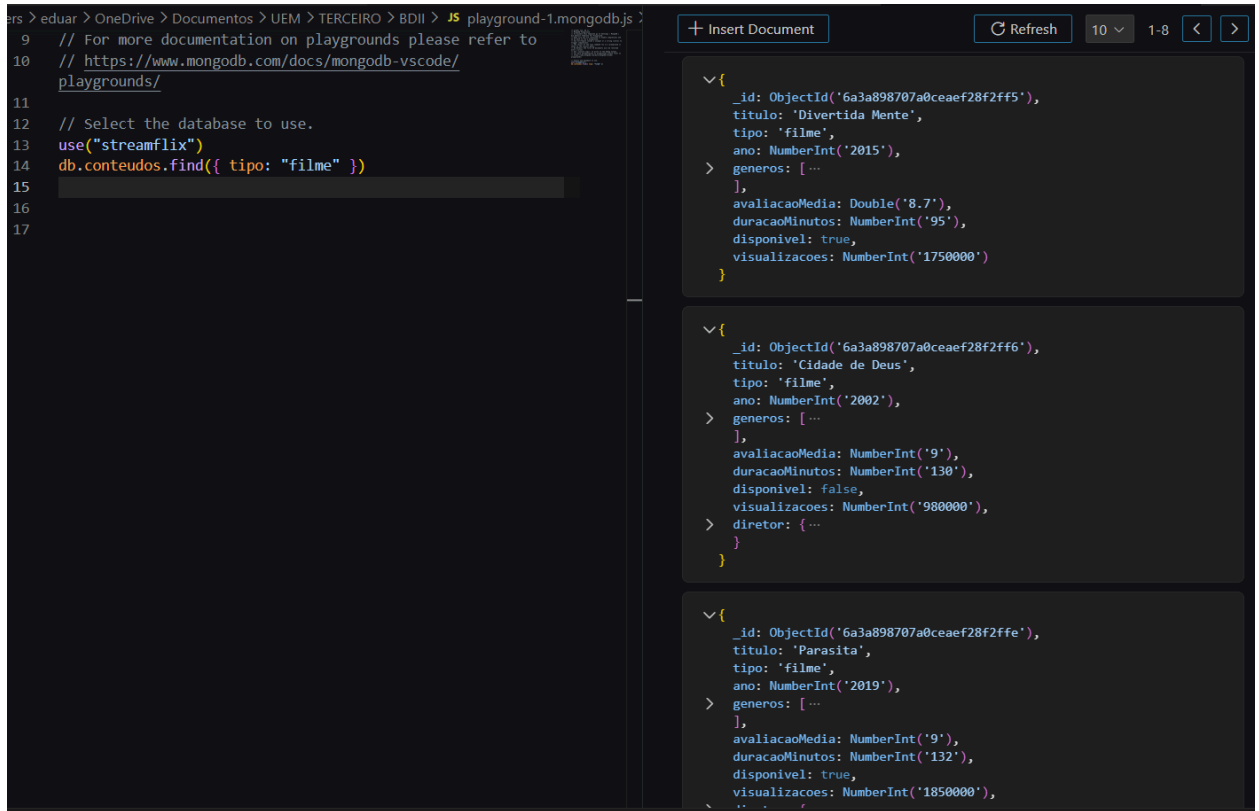
s > eduar > OneDrive > Documentos > UEM > TERCEIRO > BDII > JS playground-1.mongodb.js

```
9 // For more documentation on playgrounds please refer to
10 // https://www.mongodb.com/docs/mongodb-vscode/
11 // playgrounds/
12 // Select the database to use.
13 use("streamflix")
14 db.conteudos.find({ tipo: "filme" })
15
16
17
```

+ Insert Document Refresh 10 1-8

```
{
  _id: ObjectId('6a3a898707a0ceaf28f2ff3'),
  titulo: 'O Senhor dos Anéis: A Sociedade do Anel',
  tipo: 'filme',
  ano: NumberInt('2001'),
  generos: [ ... ],
  avaliacaoMedia: Double('9.4'),
  duracaoMinutos: NumberInt('178'),
  disponivel: true,
  visualizacoes: NumberInt('220000'),
  diretor: { ... },
  premios: [ ... ]
}
```

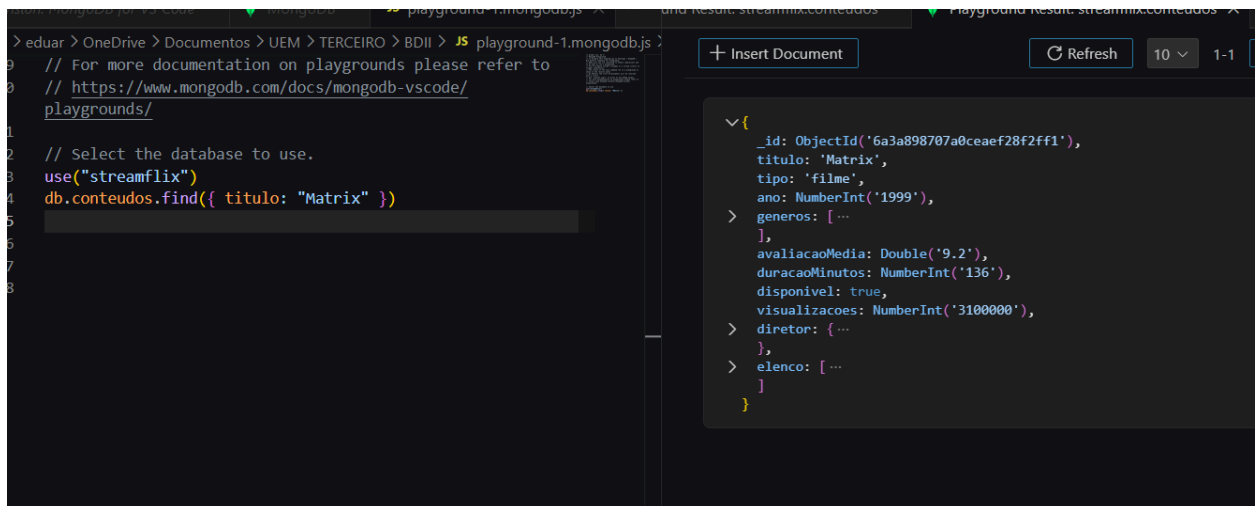
```
{
  _id: ObjectId('6a3a898707a0ceaf28f2ff4'),
  titulo: 'A Origem',
  tipo: 'filme',
  ano: NumberInt('2010'),
  generos: [ ... ],
  avaliacaoMedia: Double('9.1'),
  duracaoMinutos: NumberInt('148'),
  disponivel: true,
  visualizacoes: NumberInt('195000'),
  diretor: { ... }
}
```



Exercício 5

Busque o conteúdo cujo título é Matrix.

R: `db.conteudos.find({ titulo: "Matrix" })`



Exercício 6

Insira um novo usuário na coleção `usuarios` com os campos:

- `nome`;

- email;
- idade;
- cidade;
- estado;
- interesses;
- Ativo.

```
R: db.usuarios.insertOne({
    nome: "Eduarda Barragan",
    email: "eduarda@email.com",
    idade: 19,
    cidade: "Maringá",
    estado: "PR",
    interesses: ["Ação", "Tecnologia", "Romance"],
    ativo: true
})
```

The screenshot shows a VS Code editor with a MongoDB Playground script on the left and its output on the right. The script uses `db.usuarios.insertOne` to add a user. The output shows the document was successfully inserted with an `acknowledged: true` status and an `insertedId`.

```
// For more documentation on playgrounds please refer to
// https://www.mongodb.com/docs/mongodb-vscode/playgrounds/

// Select the database to use.
use("streamflix")
db.usuarios.insertOne({
  nome: "Eduarda Barragan",
  email: "eduarda@email.com",
  idade: 19,
  cidade: "Maringá",
  estado: "PR",
  interesses: ["Ação", "Tecnologia", "Romance"],
  ativo: true
})
```

```
1 {
2   acknowledged: true,
3   insertedId: ObjectId('6a3a8e4abde8e249132c23bf')
4 }
```

Para consultar: `db.usuarios.find({ email: "eduarda@email.com" })`

This screenshot shows the same VS Code editor with a query script on the left. The output on the right displays the document retrieved by the `find` query, showing all the fields including the `_id`, `nome`, `email`, `idade`, `cidade`, `estado`, `interesses`, and `ativo`.

```
// For more documentation on playgrounds please refer to
// https://www.mongodb.com/docs/mongodb-vscode/playgrounds/

// Select the database to use.
use("streamflix")
db.usuarios.find({ email: "eduarda@email.com" })
```

```
{
  _id: ObjectId('6a3a8e4abde8e249132c23bf'),
  nome: 'Eduarda Barragan',
  email: 'eduarda@email.com',
  idade: NumberInt('19'),
  cidade: 'Maringá',
  estado: 'PR',
  interesses: [ ... ],
  ativo: true
}
```

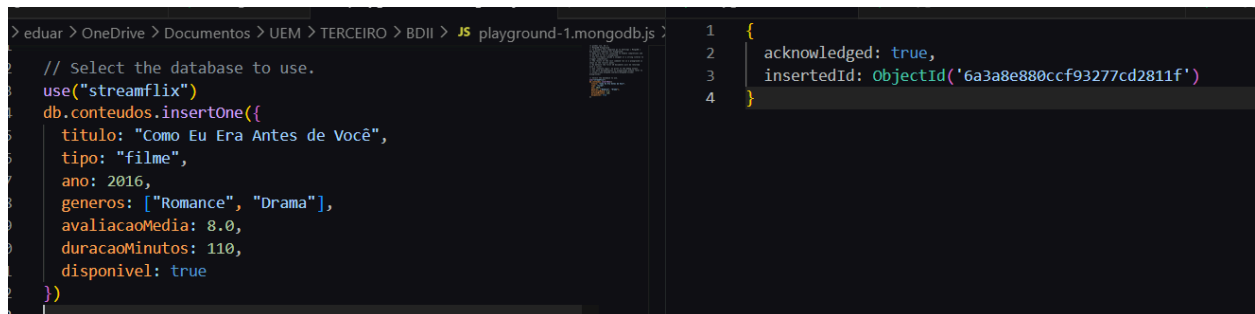
Exercício 7

Insira um novo conteúdo do tipo `filme` com os campos:

- título;

- tipo;
- ano;
- gêneros;
- avaliação média;
- duração em minutos;
- Disponível.

```
R: db.conteudos.insertOne({
    titulo: "Como Eu Era Antes de Você",
    tipo: "filme",
    ano: 2016,
    generos: ["Romance", "Drama"],
    avaliacaoMedia: 8.0,
    duracaoMinutos: 110,
    disponivel: true
})
```



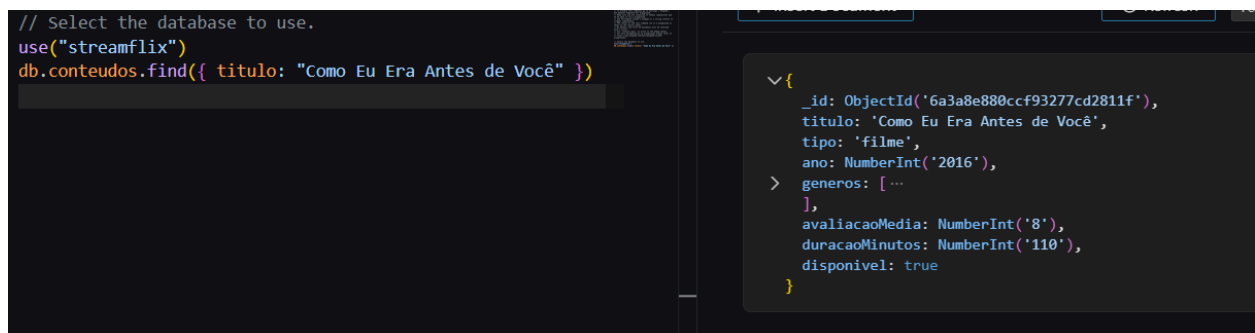
The screenshot shows a MongoDB playground interface. On the left, the code editor contains the following JavaScript code:

```
// Select the database to use.
use("streamflix")
db.conteudos.insertOne({
  titulo: "Como Eu Era Antes de Você",
  tipo: "filme",
  ano: 2016,
  generos: ["Romance", "Drama"],
  avaliacaoMedia: 8.0,
  duracaoMinutos: 110,
  disponivel: true
})
```

On the right, the output pane shows the result of the insertion:

```
{
  acknowledged: true,
  insertedId: ObjectId('6a3a8e880ccf93277cd2811f')
}
```

Para consultar: `db.conteudos.find({ titulo: "Como Eu Era Antes de Você" })`



The screenshot shows the same MongoDB playground interface. The code editor now contains the following JavaScript code:

```
// Select the database to use.
use("streamflix")
db.conteudos.find({ titulo: "Como Eu Era Antes de Você" })
```

The output pane shows the result of the query:

```
{
  _id: ObjectId('6a3a8e880ccf93277cd2811f'),
  titulo: 'Como Eu Era Antes de Você',
  tipo: 'filme',
  ano: NumberInt('2016'),
  generos: [ ... ],
  avaliacaoMedia: NumberInt('8'),
  duracaoMinutos: NumberInt('110'),
  disponivel: true
}
```

3.2. Operadores de comparação

Exercício 8

Liste os conteúdos com avaliação média maior que 9.

```
R: db.conteudos.find({
    avaliacaoMedia: { $gt: 9 }
})
```

```
// Select the database to use.
use("streamflix")
db.conteudos.find({
  avaliacaoMedia: { $gt: 9 }
})
```

```
{
  _id: ObjectId('6a3a898707a0ceaef28f2ff0'),
  titulo: 'Interestelar',
  tipo: 'filme',
  ano: NumberInt('2014'),
  generos: [ ... ],
  avaliacaoMedia: Double('9.5'),
  duracaoMinutos: NumberInt('169'),
  disponivel: true,
  visualizacoes: NumberInt('2500000'),
  diretor: { ... },
  elenco: [ ... ]
}
```

```
{
  _id: ObjectId('6a3a898707a0ceaef28f2ff1'),
  titulo: 'Matrix',
  tipo: 'filme',
  ano: NumberInt('1999'),
  generos: [ ... ],
  avaliacaoMedia: Double('9.2'),
  duracaoMinutos: NumberInt('136'),
  disponivel: true,
  visualizacoes: NumberInt('3100000'),
  diretor: { ... },
  elenco: [ ... ]
}
```

// <https://www.mongodb.com/docs/mongodb-vscode/playgrounds/>

```
// Select the database to use.
use("streamflix")
db.conteudos.find({
  avaliacaoMedia: { $gt: 9 }
})
```

+ Insert Document

Refresh

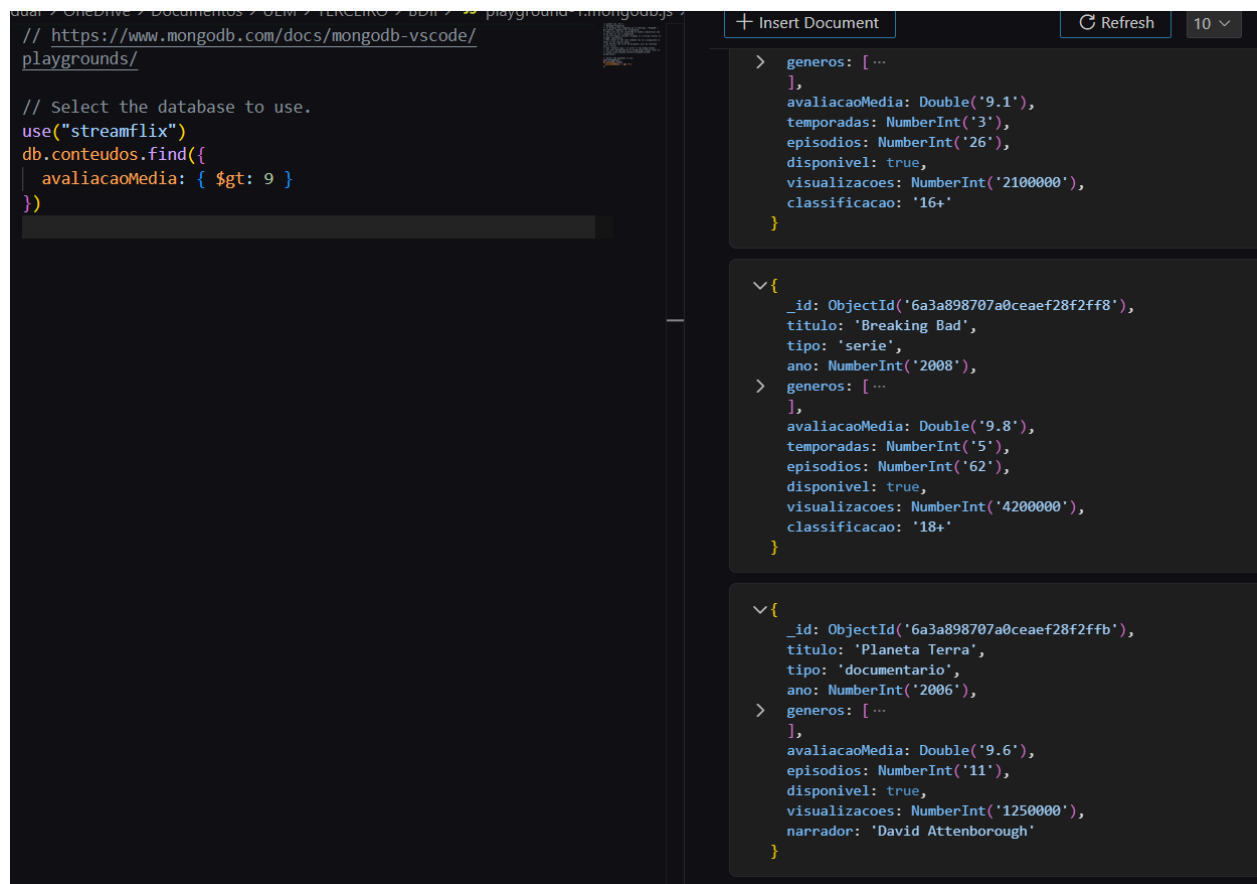
10

1-

```
{
  _id: ObjectId('6a3a898707a0ceaef28f2ff3'),
  titulo: 'O Senhor dos Anéis: A Sociedade do Anel',
  tipo: 'filme',
  ano: NumberInt('2001'),
  > generos: [ ...
  ],
  avaliacaoMedia: Double('9.4'),
  duracaoMinutos: NumberInt('178'),
  disponivel: true,
  visualizacoes: NumberInt('220000'),
  > diretor: { ...
  },
  > premios: [ ...
  ]
}
```

```
{
  _id: ObjectId('6a3a898707a0ceaef28f2ff4'),
  titulo: 'A Origem',
  tipo: 'filme',
  ano: NumberInt('2010'),
  > generos: [ ...
  ],
  avaliacaoMedia: Double('9.1'),
  duracaoMinutos: NumberInt('148'),
  disponivel: true,
  visualizacoes: NumberInt('195000'),
  > diretor: { ...
  }
}
```

```
{
  _id: ObjectId('6a3a898707a0ceaef28f2ff7'),
  titulo: 'Dark',
  tipo: 'serie',
  ano: NumberInt('2017'),
  > generos: [ ...
  ],
  avaliacaoMedia: Double('9.1'),
  temporadas: NumberInt('3')
}
```



Exercício 9

Liste os usuários com idade maior que 30.

```
R: db.usuarios.find({
  idade: { $gt: 30 }
})
```

eduar > OneDrive > Documentos > UEM > TERCEIRO > BDII > JS playground-1.mongodb.js

```
// https://www.mongodb.com/docs/mongodb-vscode/playgrounds/

// Select the database to use.
use("streamflix")
db.usuarios.find({
  idade: { $gt: 30 }
})
```

+ Insert Document

Refresh 10 1-5

```
{
  _id: ObjectId('6a3a8911551fbf74bf9873df'),
  nome: 'Carlos Lima',
  email: 'carlos@email.com',
  idade: NumberInt('31'),
  cidade: 'Maringá',
  estado: 'PR',
  interesses: [ ... ],
  ativo: false,
  endereco: { ... }
}
```

```
{
  _id: ObjectId('6a3a8911551fbf74bf9873e2'),
  nome: 'Marina Costa',
  email: 'marina@email.com',
  idade: NumberInt('35'),
  cidade: 'São Paulo',
  estado: 'SP',
  interesses: [ ... ],
  ativo: true,
  premium: true
}
```

```
{
  _id: ObjectId('6a3a8911551fbf74bf9873e3'),
  nome: 'Rafael Oliveira',
  email: 'rafael@email.com',
  idade: NumberInt('42'),
  cidade: 'Rio de Janeiro',
  estado: 'RJ',
  interesses: [ ... ],
  ativo: false
}
```

```
// https://www.mongodb.com/docs/mongodb-vscode/playgrounds/

// Select the database to use.
use("streamflix")
db.usuarios.find({
  idade: { $gt: 30 }
})
```

```
{
  _id: ObjectId('6a3a8911551fbf74bf9873e3'),
  nome: 'Rafael Oliveira',
  email: 'rafael@email.com',
  idade: NumberInt('42'),
  cidade: 'Rio de Janeiro',
  estado: 'RJ',
  interesses: [ ... ],
  ativo: false
}
```

```
{
  _id: ObjectId('6a3a8911551fbf74bf9873e6'),
  nome: 'Patrícia Alves',
  email: 'patricia@email.com',
  idade: NumberInt('38'),
  cidade: 'Belo Horizonte',
  estado: 'MG',
  interesses: [ ... ],
  ativo: true
}
```

```
{
  _id: ObjectId('6a3a8911551fbf74bf9873e7'),
  nome: 'Eduardo Martins',
  email: 'eduardo@email.com',
  idade: NumberInt('33'),
  cidade: 'Curitiba',
  estado: 'PR',
  interesses: [ ... ],
  ativo: true,
  premium: false
}
```


Exercício 10

Liste os conteúdos lançados antes do ano 2010.

```
R: db.conteudos.find({
  ano: { $lt: 2010 }
})
```

The screenshot shows the MongoDB VS Code playground interface. On the left, the command window displays the following code:

```
// https://www.mongodb.com/docs/mongodb-vscode/playgrounds/

// Select the database to use.
use("streamflix")
db.conteudos.find({
  ano: { $lt: 2010 }
})
```

On the right, the results panel shows three documents returned by the query:

```
{
  "_id": ObjectId("6a3a898707a0ceaf28f2ff1"),
  "titulo": "Matrix",
  "tipo": "filme",
  "ano": NumberInt("1999"),
  "generos": [...],
  "avaliacaoMedia": Double("9.2"),
  "duracaoMinutos": NumberInt("136"),
  "disponivel": true,
  "visualizacoes": NumberInt("3100000"),
  "diretor": {...},
  "elenco": [...],
}
```

```
{
  "_id": ObjectId("6a3a898707a0ceaf28f2ff2"),
  "titulo": "Avatar",
  "tipo": "filme",
  "ano": NumberInt("2009"),
  "generos": [...],
  "avaliacaoMedia": Double("8.8"),
  "duracaoMinutos": NumberInt("162"),
  "disponivel": true,
  "visualizacoes": NumberInt("2800000"),
  "diretor": {...},
}
```

```
{
  "_id": ObjectId("6a3a898707a0ceaf28f2ff3"),
  "titulo": "O Senhor dos Anéis: A Sociedade do Anel",
  "tipo": "filme",
  "ano": NumberInt("2001"),
  "generos": [...],
}
```

The screenshot shows the MongoDB VS Code interface. On the left, the MongoDB Shell contains the following code:

```
// https://www.mongodb.com/docs/mongodb-vscode/playgrounds/  
  
// Select the database to use.  
use("streamflix")  
db.conteudos.find({  
  ano: { $lt: 2010 }  
})
```

On the right, the 'Insert Document' panel displays three JSON documents from the 'streamflix' database, filtered by the query. The documents are:

- Document 1:**

```
{  
  "_id": ObjectId('6a3a898707a0ceaef28f2ff6'),  
  "titulo": 'Cidade de Deus',  
  "tipo": 'filme',  
  "ano": NumberInt('2002'),  
  "generos": [...],  
  "avaliacaoMedia": NumberInt('9'),  
  "duracaoMinutos": NumberInt('130'),  
  "disponivel": false,  
  "visualizacoes": NumberInt('980000'),  
  "diretor": {...}  
}
```
- Document 2:**

```
{  
  "_id": ObjectId('6a3a898707a0ceaef28f2ff8'),  
  "titulo": 'Breaking Bad',  
  "tipo": 'serie',  
  "ano": NumberInt('2008'),  
  "generos": [...],  
  "avaliacaoMedia": Double('9.8'),  
  "temporadas": NumberInt('5'),  
  "episodios": NumberInt('62'),  
  "disponivel": true,  
  "visualizacoes": NumberInt('4200000'),  
  "classificacao": '18+'  
}
```
- Document 3:**

```
{  
  "_id": ObjectId('6a3a898707a0ceaef28f2ffa'),  
  "titulo": 'The Office',  
  "tipo": 'serie',  
  "ano": NumberInt('2005'),  
  "generos": [...],  
  "avaliacaoMedia": Double('8.8'),  
  "temporadas": NumberInt('9'),  
  "episodios": NumberInt('201'),  
  "disponivel": true,  
  "classificacao": '12+'  
}
```

The bottom part of the image shows a continuation of the results, including a document for 'Planeta Terra'.

Exercício 11

Liste os conteúdos lançados a partir de 2015.

```
R: db.conteudos.find({  
  ano: { $gte: 2015 }  
})
```

qual > OneDrive > Documentos > GEM > TERCEIRO > BDII > playground-1.mongodbjs >
// <https://www.mongodb.com/docs/mongodb-vscode/playgrounds/>

```
// Select the database to use.  
use("streamflix")  
db.conteudos.find({  
  ano: { $gte: 2015 }  
})
```

+ Insert Document

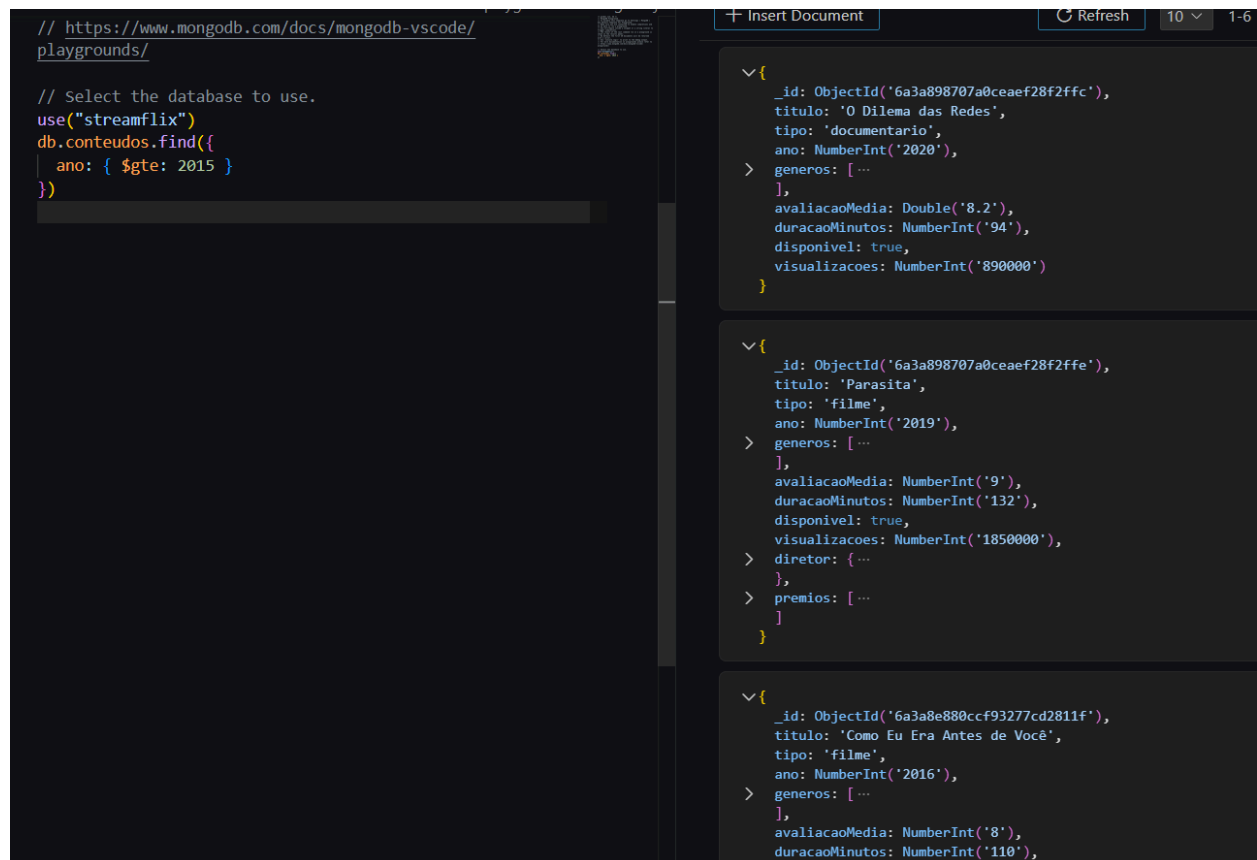
Refresh

10

```
{  
  _id: ObjectId('6a3a898707a0ceaef28f2ff5'),  
  titulo: 'Divertida Mente',  
  tipo: 'filme',  
  ano: NumberInt('2015'),  
  generos: [...]  
},  
  avaliacaoMedia: Double('8.7'),  
  duracaoMinutos: NumberInt('95'),  
  disponivel: true,  
  visualizacoes: NumberInt('1750000')  
}
```

```
{  
  _id: ObjectId('6a3a898707a0ceaef28f2ff7'),  
  titulo: 'Dark',  
  tipo: 'serie',  
  ano: NumberInt('2017'),  
  generos: [...]  
},  
  avaliacaoMedia: Double('9.1'),  
  temporadas: NumberInt('3'),  
  episodios: NumberInt('26'),  
  disponivel: true,  
  visualizacoes: NumberInt('2100000'),  
  classificacao: '16+'  
}
```

```
{  
  _id: ObjectId('6a3a898707a0ceaef28f2ff9'),  
  titulo: 'Stranger Things',  
  tipo: 'serie',  
  ano: NumberInt('2016'),  
  generos: [...]  
},  
  avaliacaoMedia: Double('8.9'),  
  temporadas: NumberInt('4'),  
  episodios: NumberInt('34'),  
  disponivel: true  
}
```



Exercício 12

Liste os conteúdos cuja avaliação média seja menor ou igual a 8.8.

```
R: db.conteudos.find({  
  avaliacaoMedia: { $lte: 8.8 }  
})
```

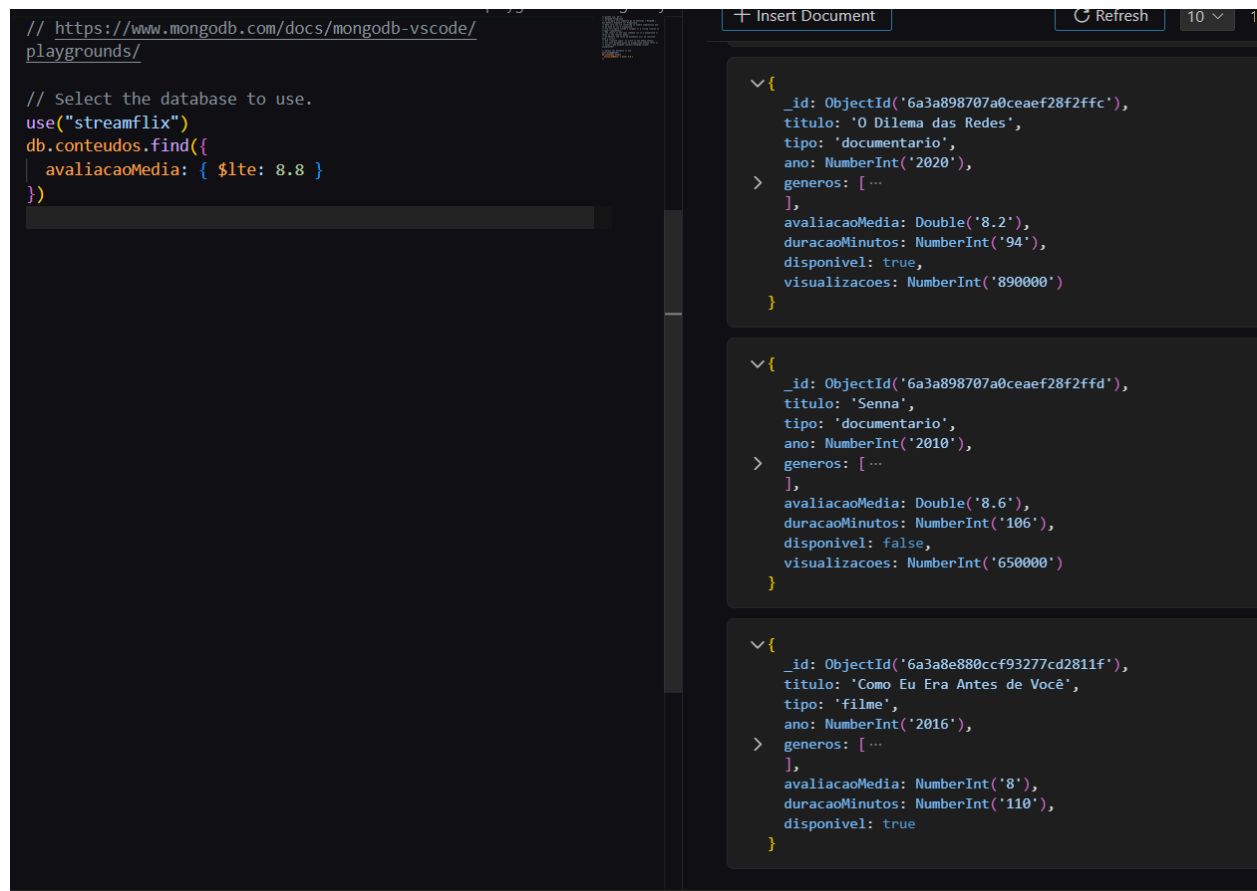
```
// https://www.mongodb.com/docs/mongodb-vscode/playgrounds/
```

```
// Select the database to use.  
use("streamflix")  
db.conteudos.find({  
  avaliacaoMedia: { $lte: 8.8 }  
})
```

```
✓{  
  _id: ObjectId('6a3a898707a0ceaf28f2ff2'),  
  titulo: 'Avatar',  
  tipo: 'filme',  
  ano: NumberInt('2009'),  
  > generos: [ ...  
    ],  
  avaliacaoMedia: Double('8.8'),  
  duracaoMinutos: NumberInt('162'),  
  disponivel: true,  
  visualizacoes: NumberInt('280000'),  
  > diretor: { ...  
    }  
  }  
}
```

```
✓{  
  _id: ObjectId('6a3a898707a0ceaf28f2ff5'),  
  titulo: 'Divertida Mente',  
  tipo: 'filme',  
  ano: NumberInt('2015'),  
  > generos: [ ...  
    ],  
  avaliacaoMedia: Double('8.7'),  
  duracaoMinutos: NumberInt('95'),  
  disponivel: true,  
  visualizacoes: NumberInt('175000')  
  }  
}
```

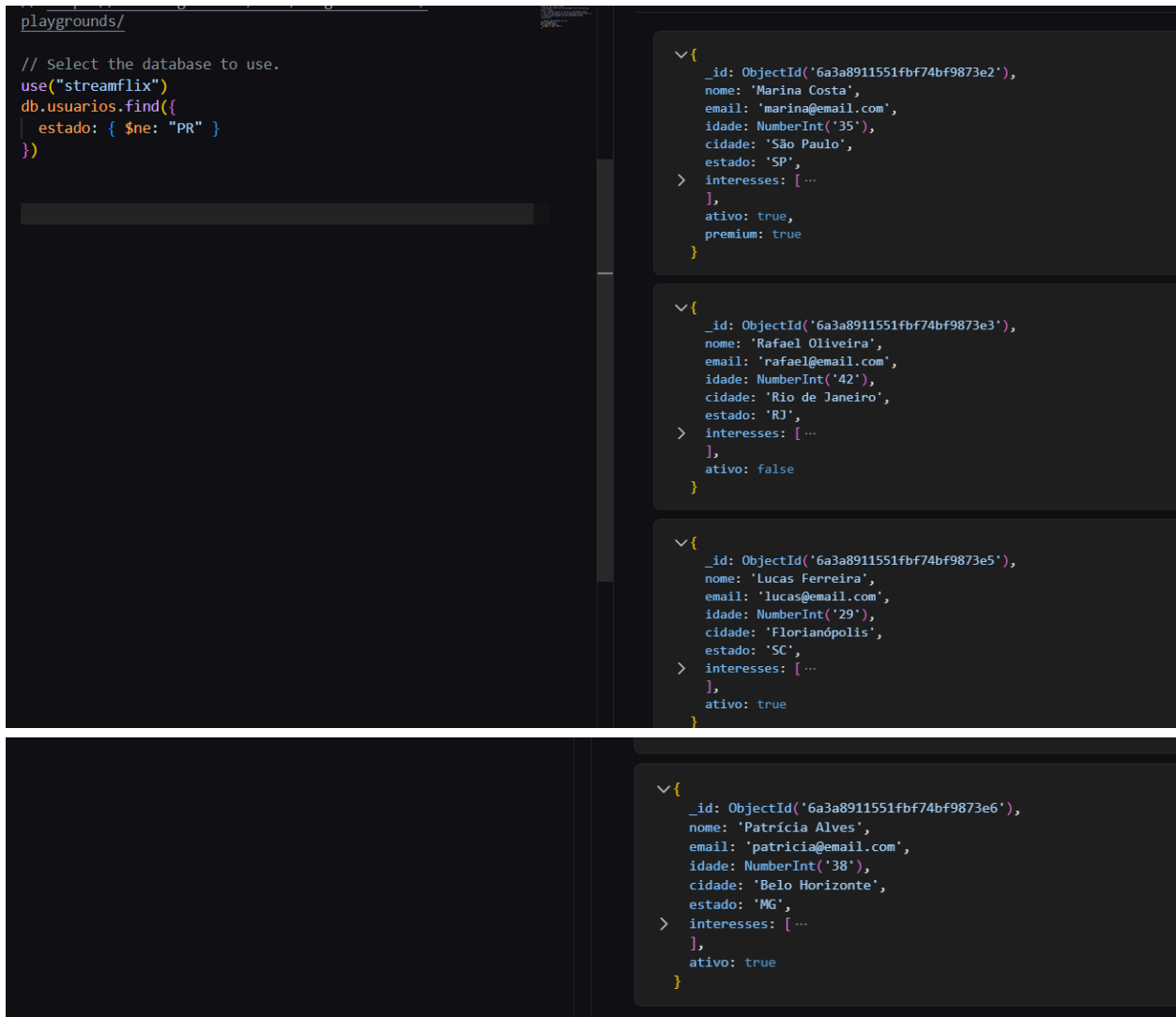
```
✓{  
  _id: ObjectId('6a3a898707a0ceaf28f2ffa'),  
  titulo: 'The Office',  
  tipo: 'serie',  
  ano: NumberInt('2005'),  
  > generos: [ ...  
    ],  
  avaliacaoMedia: Double('8.8'),  
  temporadas: NumberInt('9'),  
  episodios: NumberInt('201'),  
  }  
}
```



Exercício 13

Liste os usuários que não são do estado PR.

```
R: db.usuarios.find(  
  estado: { $ne: "PR" }  
)
```



3.3. Consultas com arrays

Exercício 14

Liste os conteúdos que possuem o gênero Drama.

```
R: db.conteudos.find(  
  | generos: "Drama"  
  | )  
})
```

uar > OneDrive > Documentos > UEM > TERCEIRO > BDII > JS playground-1.mongodb.js >
// https://www.mongodb.com/docs/mongodb-vscode/playgrounds/

```
// Select the database to use.  
use("streamflix")  
db.conteudos.find({  
  generos: "Drama"  
})
```

+ Insert Document Refresh 10 1-6

```
{  
  _id: ObjectId('6a3a898707a0ceaef28f2ff0'),  
  titulo: 'Interestelar',  
  tipo: 'filme',  
  ano: NumberInt('2014'),  
  generos: [  
    'Drama',  
    'Ficção Científica'  
  ],  
  avaliacaoMedia: Double('9.5'),  
  duracaoMinutos: NumberInt('169'),  
  disponivel: true,  
  visualizacoes: NumberInt('2500000'),  
  diretor: { ... },  
  elenco: [ ... ]  
}
```

```
{  
  _id: ObjectId('6a3a898707a0ceaef28f2ff6'),  
  titulo: 'Cidade de Deus',  
  tipo: 'filme',  
  ano: NumberInt('2002'),  
  generos: [  
    'Drama',  
    'Crime'  
  ],  
  avaliacaoMedia: NumberInt('9'),  
  duracaoMinutos: NumberInt('130'),  
  disponivel: false,  
  visualizacoes: NumberInt('980000'),  
  diretor: { ... }  
}
```

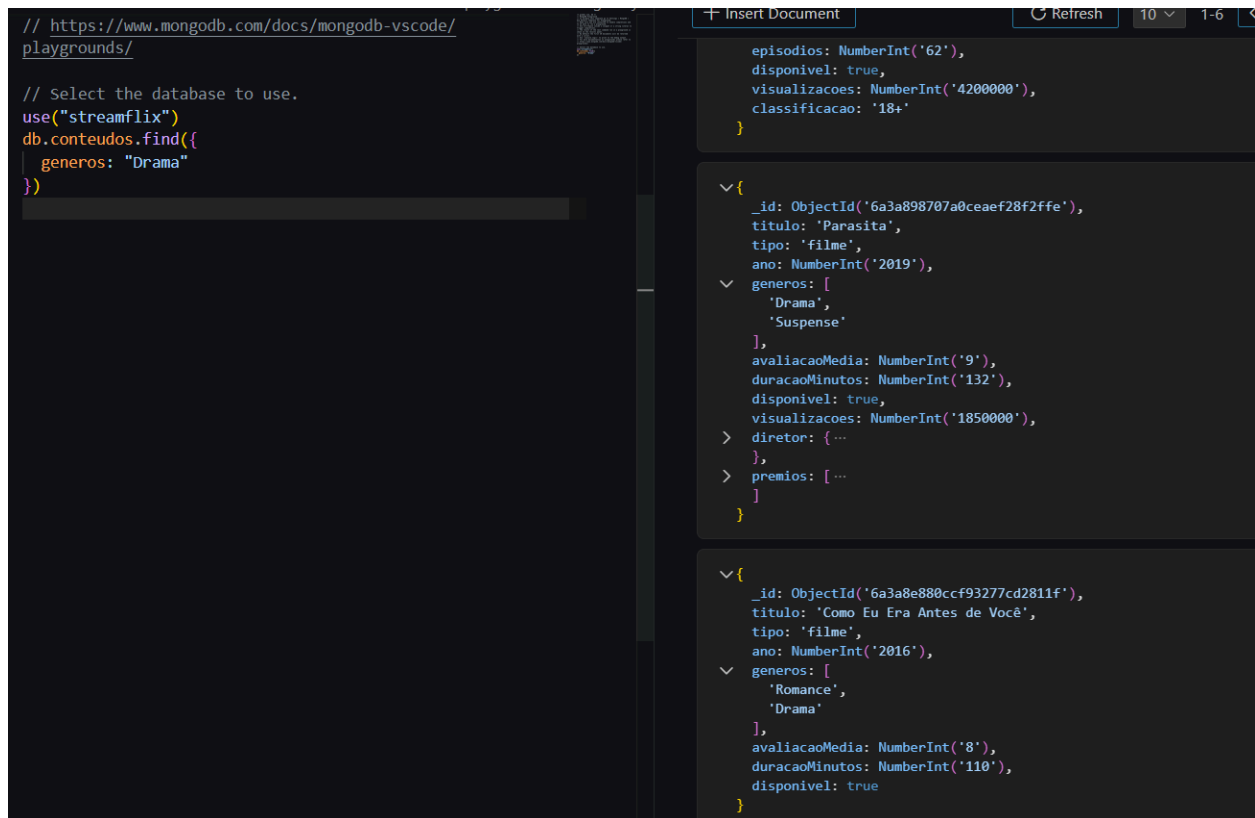
eduar > OneDrive > Documentos > UEM > TERCEIRO > BDII > JS playground-1.mongodb.js >
// https://www.mongodb.com/docs/mongodb-vscode/playgrounds/

```
// Select the database to use.  
use("streamflix")  
db.conteudos.find({  
  generos: "Drama"  
})
```

+ Insert Document Refresh 10 1-6

```
{  
  _id: ObjectId('6a3a898707a0ceaef28f2ff7'),  
  titulo: 'Dark',  
  tipo: 'serie',  
  ano: NumberInt('2017'),  
  generos: [  
    'Mistério',  
    'Ficção Científica',  
    'Drama'  
  ],  
  avaliacaoMedia: Double('9.1'),  
  temporadas: NumberInt('3'),  
  episodios: NumberInt('26'),  
  disponivel: true,  
  visualizacoes: NumberInt('2100000'),  
  classificacao: '16+'  
}
```

```
{  
  _id: ObjectId('6a3a898707a0ceaef28f2ff8'),  
  titulo: 'Breaking Bad',  
  tipo: 'serie',  
  ano: NumberInt('2008'),  
  generos: [  
    'Drama',  
    'Crime'  
  ],  
  avaliacaoMedia: Double('9.8'),  
  temporadas: NumberInt('5'),  
  episodios: NumberInt('62'),  
  disponivel: true,  
  visualizacoes: NumberInt('4200000'),  
  classificacao: '18+'  
}
```

Exercício 15

Liste os conteúdos que possuem o gênero Ficção Científica.

```
R: db.conteudos.find({  
  generos: "Ficção Científica"  
})
```

```
eduar > OneDrive > Documentos > UEM > TERCEIRO > BDII > JS playground-1.mongodb.js >
// https://www.mongodb.com/docs/mongodb-vscode/playgrounds/

// Select the database to use.
use("streamflix")
db.conteudos.find({
  generos: "Ficção Científica"
})
```

+ Insert Document

Refresh

10

```
{
  "_id": ObjectId("6a3a898707a0ceaef28f2ff0"),
  "titulo": "Interestelar",
  "tipo": "filme",
  "ano": NumberInt("2014"),
  "generos": [
    "Drama",
    "Ficção Científica"
  ],
  "avaliacaoMedia": Double("9.5"),
  "duracaoMinutos": NumberInt("169"),
  "disponivel": true,
  "visualizacoes": NumberInt("250000"),
  "diretor": {
  },
  "elenco": [
  ]
}
```

```
{
  "_id": ObjectId("6a3a898707a0ceaef28f2ff1"),
  "titulo": "Matrix",
  "tipo": "filme",
  "ano": NumberInt("1999"),
  "generos": [
    "Ação",
    "Ficção Científica"
  ],
  "avaliacaoMedia": Double("9.2"),
  "duracaoMinutos": NumberInt("136"),
  "disponivel": true,
  "visualizacoes": NumberInt("310000"),
  "diretor": {
  },
  "elenco": [
  ]
}
```

```
eduar > OneDrive > Documentos > UEM > TERCEIRO > BDII > JS playground-1.mongodb.js >
// https://www.mongodb.com/docs/mongodb-vscode/playgrounds/

// Select the database to use.
use("streamflix")
db.conteudos.find({
  generos: "Ficção Científica"
})
```

+ Insert Document

Refresh

10

1-6

```
{
  "_id": ObjectId("6a3a898707a0ceaef28f2ff2"),
  "titulo": "Avatar",
  "tipo": "filme",
  "ano": NumberInt("2009"),
  "generos": [
    "Aventura",
    "Ficção Científica"
  ],
  "avaliacaoMedia": Double("8.8"),
  "duracaoMinutos": NumberInt("162"),
  "disponivel": true,
  "visualizacoes": NumberInt("280000"),
  "diretor": {
  }
}
```

```
{
  "_id": ObjectId("6a3a898707a0ceaef28f2ff4"),
  "titulo": "A Origem",
  "tipo": "filme",
  "ano": NumberInt("2010"),
  "generos": [
    "Ação",
    "Ficção Científica",
    "Suspense"
  ],
  "avaliacaoMedia": Double("9.1"),
  "duracaoMinutos": NumberInt("148"),
  "disponivel": true,
  "visualizacoes": NumberInt("195000"),
  "diretor": {
  }
}
```

```
// https://www.mongodb.com/docs/mongodb-vscode/playgrounds/

// Select the database to use.
use("streamflix")
db.conteudos.find({
  generos: "Ficção Científica"
})
```

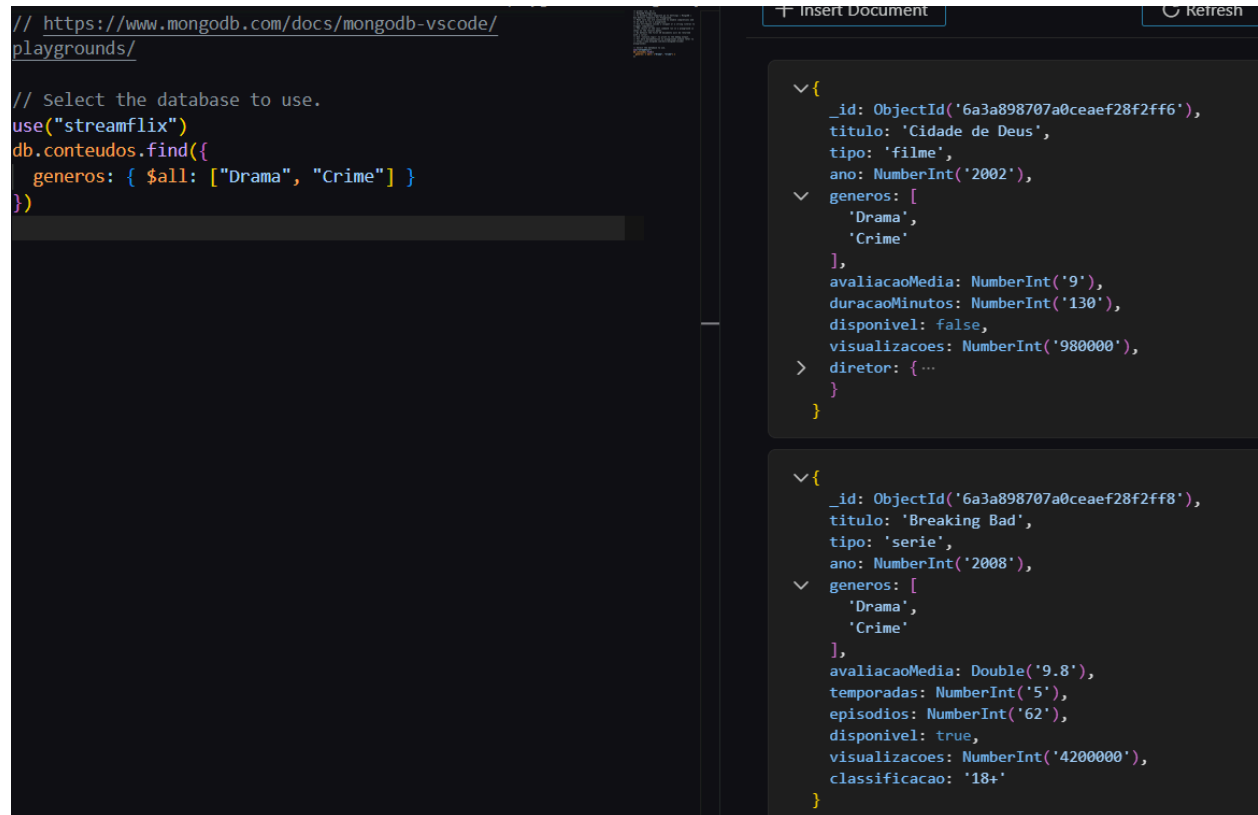
```
{
  "_id": "6a3a898707a0ceaef28f2ff7",
  "titulo": "Dark",
  "tipo": "serie",
  "ano": 2017,
  "generos": [
    "Mistério",
    "Ficção Científica",
    "Drama"
  ],
  "avaliacaoMedia": 9.1,
  "temporadas": 3,
  "episodios": 26,
  "disponivel": true,
  "visualizacoes": 2100000,
  "classificacao": "16+"
}
```

```
{
  "_id": "6a3a898707a0ceaef28f2ff9",
  "titulo": "Stranger Things",
  "tipo": "serie",
  "ano": 2016,
  "generos": [
    "Ficção Científica",
    "Terror",
    "Aventura"
  ],
  "avaliacaoMedia": 8.9,
  "temporadas": 4,
  "episodios": 34,
  "disponivel": true,
  "visualizacoes": 3900000,
  "classificacao": "14+"
}
```

Exercício 16

Liste os conteúdos que possuem os gêneros Drama e Crime ao mesmo tempo.

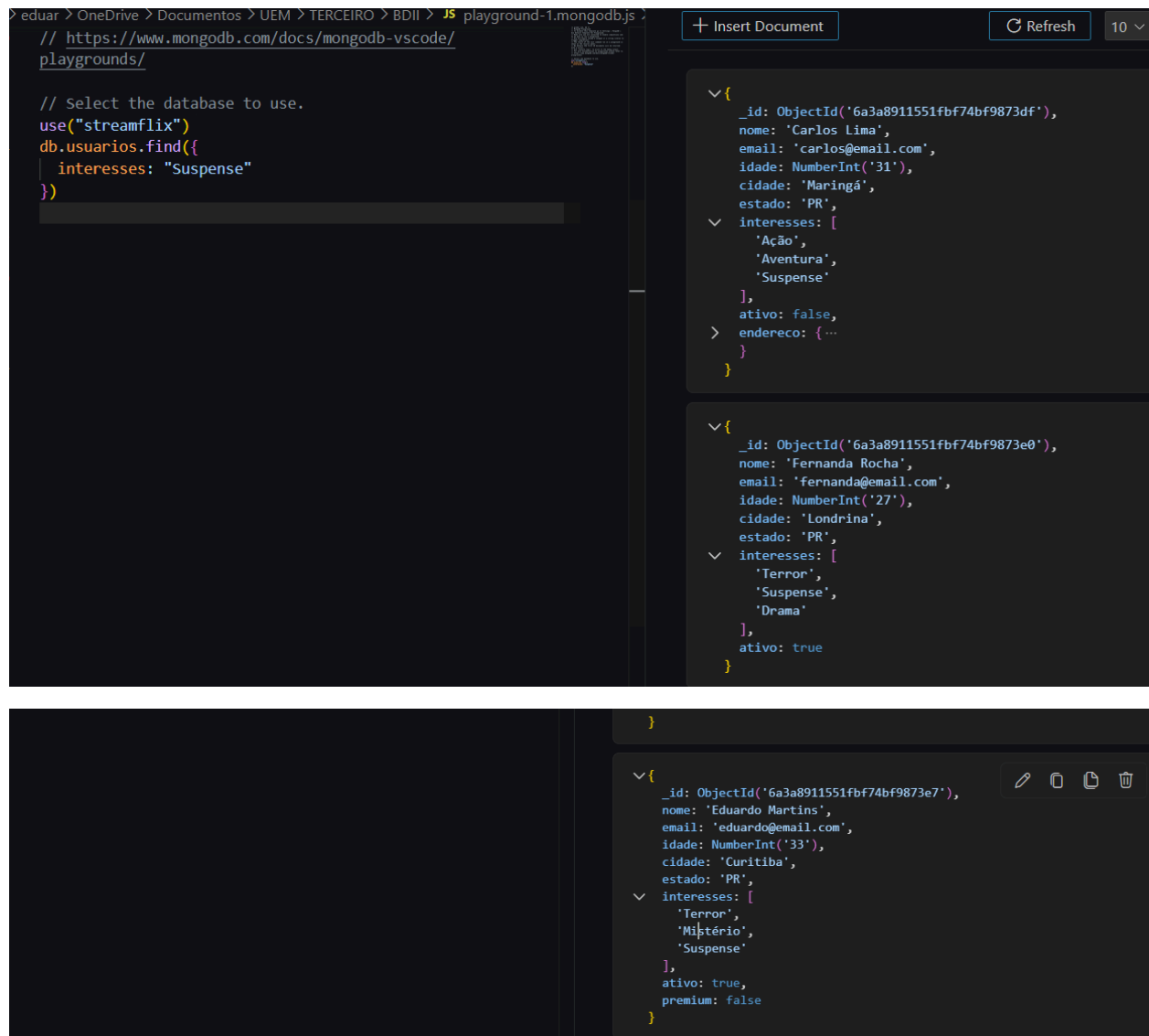
```
R: db.conteudos.find({
  generos: { $all: ["Drama", "Crime"] }
})
```



Exercício 17

Liste os usuários que possuem interesse em Suspense.

```
R: db.usuarios.find({  
  interesses: "Suspense"  
})
```



Exercício 18

Liste os conteúdos que possuem pelo menos um dos seguintes gêneros:

- Terror
- Mistério

```
db.conteudos.find({  
  generos: { $in: ["Terror", "Mistério"] }  
})
```

```
// Select the database to use.
use("streamflix")
db.conteudos.find({
  generos: { $in: ["Terror", "Mistério"] }
})
```

```

{
  _id: ObjectId('6a3a898707a0ceaef28f2ff7'),
  titulo: 'Dark',
  tipo: 'serie',
  ano: NumberInt('2017'),
  generos: [
    'Mistério',
    'Ficção Científica',
    'Drama'
  ],
  avaliacaoMedia: Double('9.1'),
  temporadas: NumberInt('3'),
  episodios: NumberInt('26'),
  disponivel: true,
  visualizacoes: NumberInt('2100000'),
  classificacao: '16+'
}

```

```

{
  _id: ObjectId('6a3a898707a0ceaef28f2ff9'),
  titulo: 'Stranger Things',
  tipo: 'serie',
  ano: NumberInt('2016'),
  generos: [
    'Ficção Científica',
    'Terror',
    'Aventura'
  ],
  avaliacaoMedia: Double('8.9'),
  temporadas: NumberInt('4'),
  episodios: NumberInt('34'),
  disponivel: true,
  visualizacoes: NumberInt('3900000'),
  classificacao: '14+'
}

```

Exercício 19

Liste os conteúdos que não possuem o gênero Comédia.

```
R: db.conteudos.find({
  generos: { $ne: "Comédia" }
})
```

```
// Select the database to use.
use("streamflix")
db.conteudos.find({
  generos: { $ne: "Comédia" }
})
```

```
{
  _id: ObjectId('6a3a898707a0ceaef28f2ff0'),
  titulo: 'Interestelar',
  tipo: 'filme',
  ano: NumberInt('2014'),
  generos: [ ... ],
  avaliacaoMedia: Double('9.5'),
  duracaoMinutos: NumberInt('169'),
  disponivel: true,
  visualizacoes: NumberInt('250000'),
  diretor: { ... },
  elenco: [ ... ]
}
```

```
{
  _id: ObjectId('6a3a898707a0ceaef28f2ff1'),
  titulo: 'Matrix',
  tipo: 'filme',
  ano: NumberInt('1999'),
  generos: [ ... ],
  avaliacaoMedia: Double('9.2'),
  duracaoMinutos: NumberInt('136'),
  disponivel: true,
  visualizacoes: NumberInt('310000'),
  diretor: { ... },
  elenco: [ ... ]
}
```

```
// https://www.mongodb.com/docs/mongodb-vscode/playgrounds/
```

```
// Select the database to use.
use("streamflix")
db.conteudos.find({
  generos: { $ne: "Comédia" }
})
```

```
{
  _id: ObjectId('6a3a898707a0ceaef28f2ff2'),
  titulo: 'Avatar',
  tipo: 'filme',
  ano: NumberInt('2009'),
  generos: [ ... ],
  avaliacaoMedia: Double('8.8'),
  duracaoMinutos: NumberInt('162'),
  disponivel: true,
  visualizacoes: NumberInt('280000'),
  diretor: { ... }
}
```

```
{
  _id: ObjectId('6a3a898707a0ceaef28f2ff3'),
  titulo: 'O Senhor dos Anéis: A Sociedade do Anel',
  tipo: 'filme',
  ano: NumberInt('2001'),
  generos: [ ... ],
  avaliacaoMedia: Double('9.4'),
  duracaoMinutos: NumberInt('178'),
  disponivel: true,
  visualizacoes: NumberInt('220000'),
  diretor: { ... },
  premios: [ ... ]
}
```

```
> eduar > OneDrive > Documentos > UEM > TERCEIRO > BDII > JS playground-1.mongodb.js
10 // https://www.mongodb.com/docs/mongodb-vscode/
11 playgrounds/
12
13 // Select the database to use.
14 use("streamflix")
15 db.conteudos.find({
16   generos: { $ne: "Comédia" }
17 })
18
19
20
21
22
23
24
25
26
```

+ Insert Document

Refresh 10 1-10

▼ {

_id: ObjectId('6a3a898707a0ceaef28f2ff4'),

titulo: 'A Origem',

tipo: 'filme',

ano: NumberInt('2010'),

> generos: [...

],

avaliacaoMedia: Double('9.1'),

duracaoMinutos: NumberInt('148'),

disponivel: true,

visualizacoes: NumberInt('1950000'),

> diretor: { ...

}

}

▼ {

_id: ObjectId('6a3a898707a0ceaef28f2ff6'),

titulo: 'Cidade de Deus',

tipo: 'filme',

ano: NumberInt('2002'),

> generos: [...

],

avaliacaoMedia: NumberInt('9'),

duracaoMinutos: NumberInt('130'),

disponivel: false,

visualizacoes: NumberInt('980000'),

> diretor: { ...

}

}

```
// https://www.mongodb.com/docs/mongodb-vscode/
playgrounds/

// Select the database to use.
use("streamflix")
db.conteudos.find({
  generos: { $ne: "Comédia" }
})
```

▼ {

_id: ObjectId('6a3a898707a0ceaef28f2ff7'),

titulo: 'Dark',

tipo: 'serie',

ano: NumberInt('2017'),

> generos: [...

],

avaliacaoMedia: Double('9.1'),

temporadas: NumberInt('3'),

episodios: NumberInt('26'),

disponivel: true,

visualizacoes: NumberInt('2100000'),

classificacao: '16+

}

▼ {

_id: ObjectId('6a3a898707a0ceaef28f2ff8'),

titulo: 'Breaking Bad',

tipo: 'serie',

ano: NumberInt('2008'),

> generos: [...

],

avaliacaoMedia: Double('9.8'),

temporadas: NumberInt('5'),

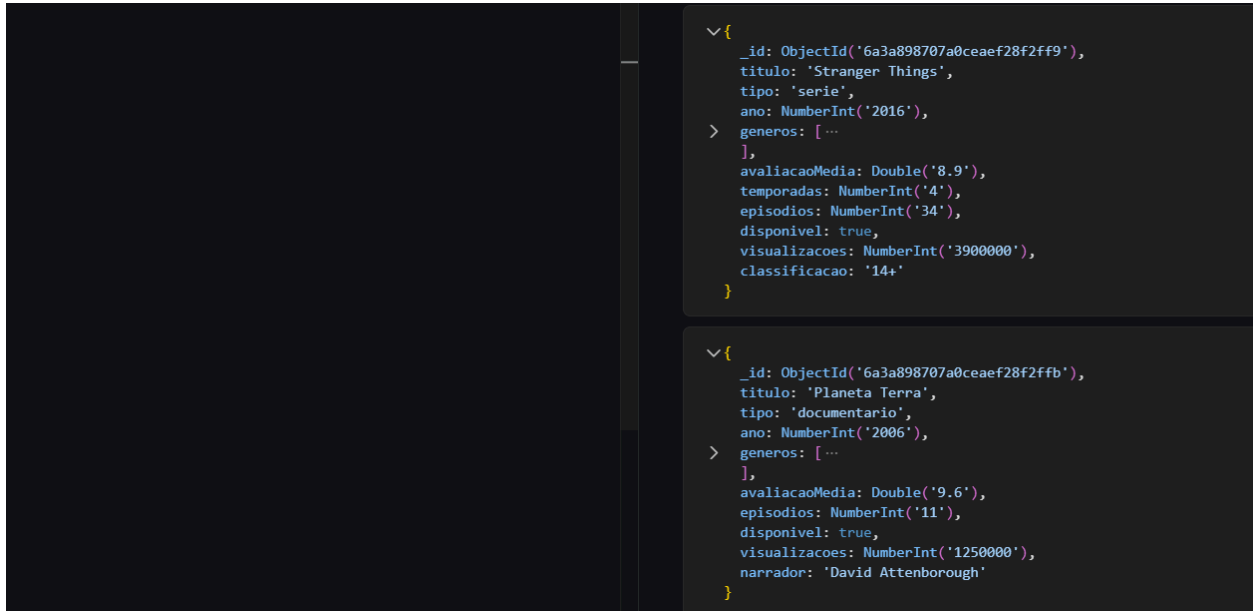
episodios: NumberInt('62'),

disponivel: true,

visualizacoes: NumberInt('4200000'),

classificacao: '18+

}



3.4. Objetos aninhados

Exercício 20

Liste os conteúdos dirigidos por Christopher Nolan.

```
R: db.conteudos.find({
  "diretor.nome": "Christopher Nolan"
})
```

```
// For more documentation on playgrounds please refer to  
// https://www.mongodb.com/docs/mongodb-vscode/  
playgrounds/
```

```
// Select the database to use.  
use("streamflix")  
db.conteudos.find({  
  "diretor.nome": "Christopher Nolan"  
})
```

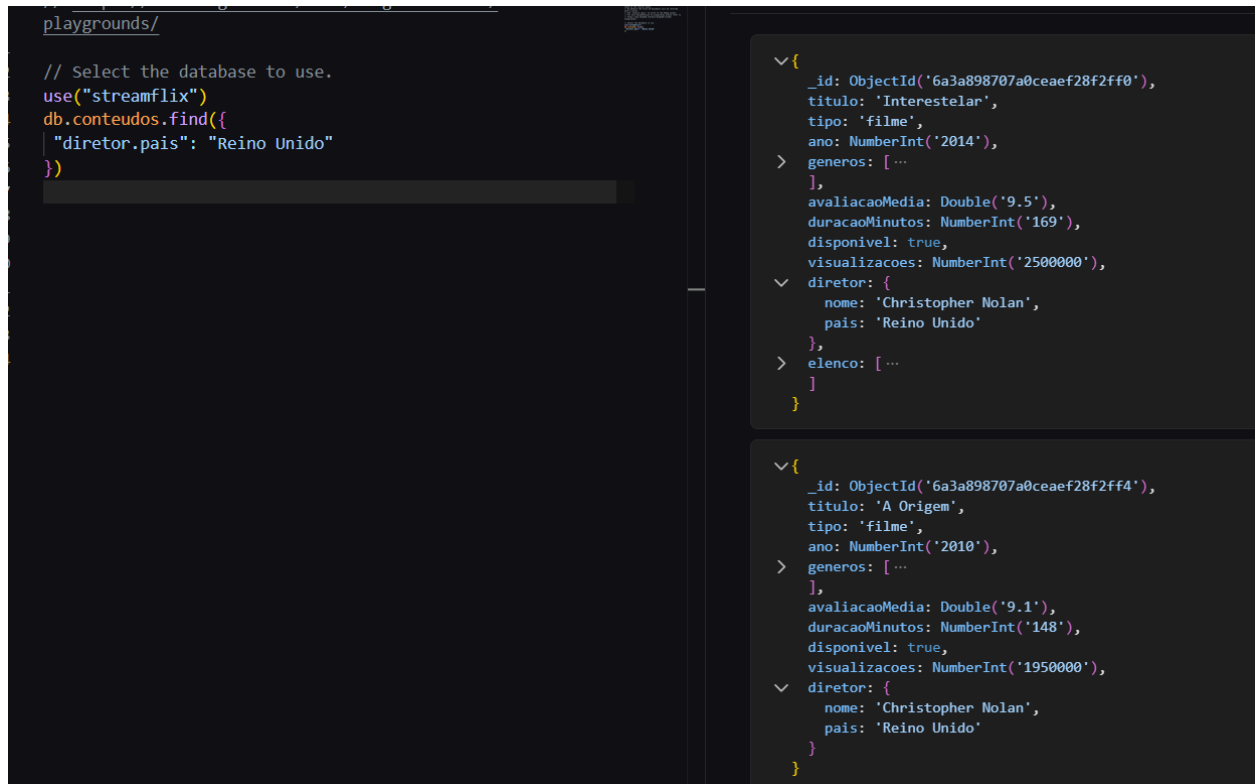
```
▼ {  
  _id: ObjectId('6a3a898707a0ceaef28f2ff0'),  
  titulo: 'Interestelar',  
  tipo: 'filme',  
  ano: NumberInt('2014'),  
  > generos: [ ...  
  ],  
  avaliacaoMedia: Double('9.5'),  
  duracaoMinutos: NumberInt('169'),  
  disponivel: true,  
  visualizacoes: NumberInt('2500000'),  
  ▼ diretor: {  
    nome: 'Christopher Nolan',  
    pais: 'Reino Unido'  
  },  
  > elenco: [ ...  
  ]  
}
```

```
▼ {  
  _id: ObjectId('6a3a898707a0ceaef28f2ff4'),  
  titulo: 'A Origem',  
  tipo: 'filme',  
  ano: NumberInt('2010'),  
  > generos: [ ...  
  ],  
  avaliacaoMedia: Double('9.1'),  
  duracaoMinutos: NumberInt('148'),  
  disponivel: true,  
  visualizacoes: NumberInt('1950000'),  
  ▼ diretor: {  
    nome: 'Christopher Nolan',  
    pais: 'Reino Unido'  
  },  
  > elenco: [ ...  
  ]  
}
```

Exercício 21

Liste os conteúdos cujo diretor é do Reino Unido.

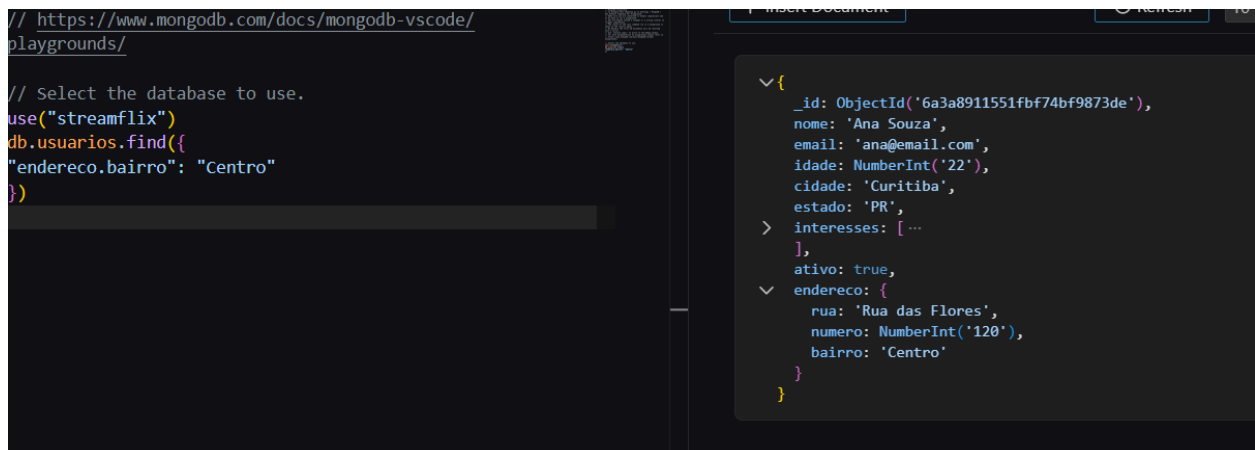
```
R: db.conteudos.find({  
  "diretor.pais": "Reino Unido"  
})
```



Exercício 22

Liste os usuários cujo bairro seja Centro.

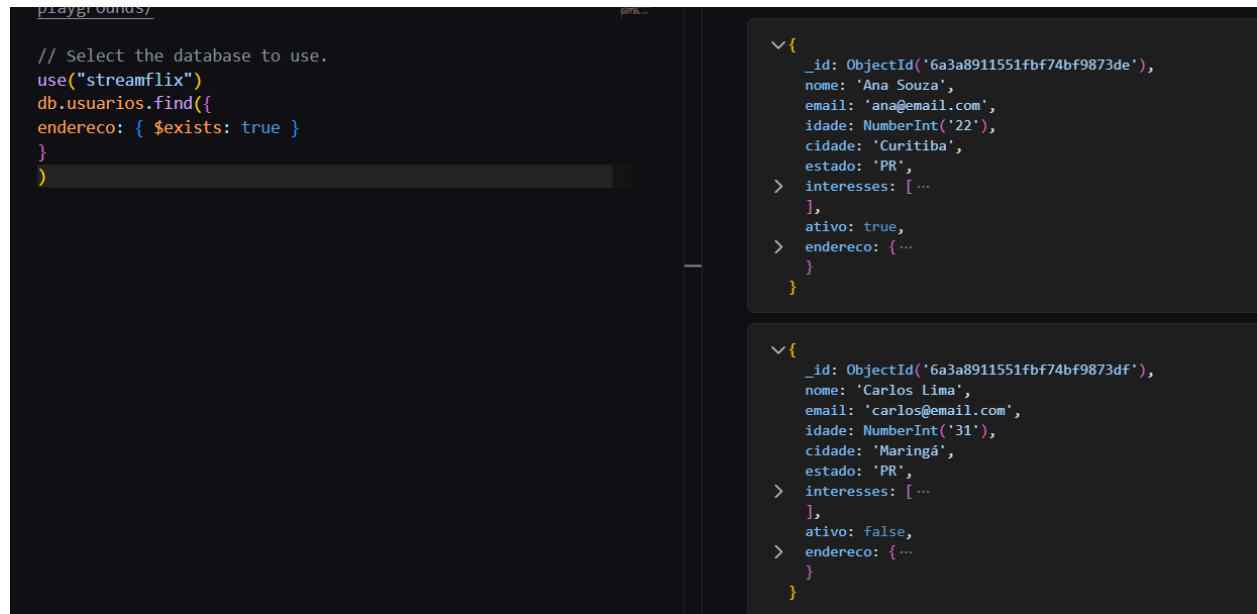
```
R: db.usuarios.find({
  "endereco.bairro": "Centro"
})
```



Exercício 23

Liste os usuários que possuem o campo `endereco`.

```
R: db.usuarios.find({
  endereco: { $exists: true }
})
```



The screenshot shows a MongoDB playground interface. On the left, the query is entered in a text area: `// Select the database to use. use("streamflix") db.usuarios.find({ endereco: { $exists: true } })`. On the right, the results are displayed in a JSON format. Two documents are shown, each with a collapsed arrow icon to its left. The first document is for 'Ana Souza' and the second is for 'Carlos Lima'. Both documents have an 'endereco' field.

```
{
  "_id": ObjectId("6a3a8911551fbf74bf9873de"),
  "nome": "Ana Souza",
  "email": "ana@email.com",
  "idade": NumberInt("22"),
  "cidade": "Curitiba",
  "estado": "PR",
  "interesses": [ ... ],
  "ativo": true,
  "endereco": { ... }
}
```

```
{
  "_id": ObjectId("6a3a8911551fbf74bf9873df"),
  "nome": "Carlos Lima",
  "email": "carlos@email.com",
  "idade": NumberInt("31"),
  "cidade": "Maringá",
  "estado": "PR",
  "interesses": [ ... ],
  "ativo": false,
  "endereco": { ... }
}
```

Exercício 24

Liste os usuários que não possuem o campo `endereco`.

```
R: db.usuarios.find({
  endereco: { $exists: false }
})
```

```
// Select the database to use.
use("streamflix")
db.usuarios.find({
  endereco: { $exists: false }
})
```

Insert Document Refresh 10 1-9

- ```
{
 _id: ObjectId('6a3a8911551fbf74bf9873e0'),
 nome: 'Fernanda Rocha',
 email: 'fernanda@email.com',
 idade: NumberInt('27'),
 cidade: 'Londrina',
 estado: 'PR',
 interesses: [...],
 ativo: true
}
```
- ```
{
  _id: ObjectId('6a3a8911551fbf74bf9873e1'),
  nome: 'João Mendes',
  email: 'joao@email.com',
  idade: NumberInt('19'),
  cidade: 'Curitiba',
  estado: 'PR',
  interesses: [ ... ],
  ativo: true
}
```
- ```
{
 _id: ObjectId('6a3a8911551fbf74bf9873e2'),
 nome: 'Marina Costa',
 email: 'marina@email.com',
 idade: NumberInt('35'),
 cidade: 'São Paulo',
 estado: 'SP',
 interesses: [...],
 ativo: true,
 premium: true
}
```

```
Equipe > OneDrive > Documentos > DEM > TERCEIRO > BDII > playground-firebase-jobs
// Select the database to use.
use("streamflix")
db.usuarios.find({
 endereco: { $exists: false }
})
```

Insert Document Refresh 10

- ```
{
  _id: ObjectId('6a3a8911551fbf74bf9873e3'),
  nome: 'Rafael Oliveira',
  email: 'rafael@email.com',
  idade: NumberInt('42'),
  cidade: 'Rio de Janeiro',
  estado: 'RJ',
  interesses: [ ... ],
  ativo: false
}
```
- ```
{
 _id: ObjectId('6a3a8911551fbf74bf9873e4'),
 nome: 'Beatriz Nunes',
 email: 'beatriz@email.com',
 idade: NumberInt('24'),
 cidade: 'Maringá',
 estado: 'PR',
 interesses: [...],
 ativo: true,
 telefone: '4499999-0000'
}
```
- ```
{
  _id: ObjectId('6a3a8911551fbf74bf9873e5'),
  nome: 'Lucas Ferreira',
  email: 'lucas@email.com',
  idade: NumberInt('29'),
  cidade: 'Florianópolis',
  estado: 'SC',
  interesses: [ ... ],
  ativo: true
}
```

```
use("streamflix")
db.usuarios.find({
  endereco: { $exists: false }
})
```

```
{
  "_id": ObjectId("6a3a8911551fbf74bf9873e6"),
  "nome": "Patrícia Alves",
  "email": "patricia@email.com",
  "idade": NumberInt("38"),
  "cidade": "Belo Horizonte",
  "estado": "MG",
  "interesses": [ ... ],
  "ativo": true
}
```

```
{
  "_id": ObjectId("6a3a8911551fbf74bf9873e7"),
  "nome": "Eduardo Martins",
  "email": "eduardo@email.com",
  "idade": NumberInt("33"),
  "cidade": "Curitiba",
  "estado": "PR",
  "interesses": [ ... ],
  "ativo": true,
  "premium": false
}
```

```
{
  "_id": ObjectId("6a3a8e4abde8e249132c23bf"),
  "nome": "Eduarda Barragan",
  "email": "eduarda@email.com",
  "idade": NumberInt("19"),
  "cidade": "Maringá",
  "estado": "PR",
  "interesses": [ ... ],
  "ativo": true
}
```

3.5. Atualizações básicas

Exercício 25

Atualize o usuário Carlos Lima para que o campo ativo passe a ser true.

```
R: db.usuarios.updateOne(
  { nome: "Carlos Lima" },
  { $set: { ativo: true } }
)
```

```
eduar > OneDrive > Documentos > UEM > TERCEIRO > BDII > JS playground-1.mongodb.js > 1 {
  2   acknowledged: true,
  3   insertedId: null,
  4   matchedCount: NumberInt('1'),
  5   modifiedCount: NumberInt('1'),
  6   upsertedCount: NumberInt('0')
  7 }
```

```
Connected to banco.zh1911.mongodb.net
// Select the database to use.
use("streamflix")
db.usuarios.updateOne(
  { nome: "Carlos Lima" },
  { $set: { ativo: true } }
)
```

Exercício 26

Atualize o conteúdo Cidade de Deus para que o campo disponivel passe a ser true.

```
R: db.conteudos.updateOne(
```

```

    { titulo: "Cidade de Deus" },
    { $set: { disponivel: true } }
  )

```

The screenshot shows the MongoDB Playground interface. On the left, the command window displays the following code:


```

// Connected to banco.zhl1911.mongodb.net
// Select the database to use.
use("streamflix")
db.conteudos.updateOne(
  { titulo: "Cidade de Deus" },
  { $set: { disponivel: true } }
)
  
```

 On the right, the 'Playground Result' tab shows the execution details:


```

1 {
2   acknowledged: true,
3   insertedId: null,
4   matchedCount: NumberInt('1'),
5   modifiedCount: NumberInt('1'),
6   upsertedCount: NumberInt('0')
7 }
  
```

Exercício 27

Adicione o campo `idiomaOriginal` ao filme `Matrix`, com o valor `Inglês`.

```

R: db.conteudos.updateOne(
  { titulo: "Matrix" },
  { $set: { idiomaOriginal: "Inglês" } }
)

```

The screenshot shows the MongoDB Playground interface. On the left, the command window displays the following code:


```

// Connected to banco.zhl1911.mongodb.net
// Select the database to use.
use("streamflix")
db.conteudos.updateOne(
  { titulo: "Matrix" },
  { $set: { idiomaOriginal: "Inglês" } }
)
  
```

 On the right, the 'Playground Result' tab shows the execution details:


```

1 {
2   acknowledged: true,
3   insertedId: null,
4   matchedCount: NumberInt('1'),
5   modifiedCount: NumberInt('1'),
6   upsertedCount: NumberInt('0')
7 }
  
```

Exercício 28

Adicione o campo `classificacao` ao filme `Interestelar`, com o valor `10+`.

```

R: db.conteudos.updateOne(
  { titulo: "Interestelar" },
  { $set: { classificacao: "10+" } }
)

```

The screenshot shows the MongoDB Playground interface. On the left, the command window displays the following code:


```

// Connected to banco.zhl1911.mongodb.net
// Select the database to use.
use("streamflix")
db.conteudos.updateOne(
  { titulo: "Interestelar" },
  { $set: { classificacao: "10+" } }
)
  
```

 On the right, the 'Playground Result' tab shows the execution details:


```

1 {
2   acknowledged: true,
3   insertedId: null,
4   matchedCount: NumberInt('1'),
5   modifiedCount: NumberInt('1'),
6   upsertedCount: NumberInt('0')
7 }
  
```

Exercício 29

Atualize a avaliação média de `Avatar` para `9.0`.

```
R: db.conteudos.updateOne(
    { titulo: "Avatar" },
    { $set: { avaliacaoMedia: 9.0 } }
)
```

The screenshot shows the MongoDB Playground interface. The left pane contains the following JavaScript code:

```
1 // Select the database to use.
2 use("streamflix")
3 db.conteudos.updateOne(
4   { titulo: "Avatar" },
5   { $set: { avaliacaoMedia: 9.0 } }
6 )
7
```

The right pane shows the result of the command:

```
1 {
2   acknowledged: true,
3   insertedId: null,
4   matchedCount: NumberInt('1'),
5   modifiedCount: NumberInt('1'),
6   upsertedCount: NumberInt('0')
7 }
```

3.6. Atualizações com operadores

Exercício 30

Incremente em 1 a quantidade de visualizações do conteúdo `Matrix`.

```
R: db.conteudos.updateOne(
    { titulo: "Matrix" },
    { $inc: { visualizacoes: 1 } }
)
```

The screenshot shows the MongoDB Playground interface. The left pane contains the following JavaScript code:

```
1 // Connected to banco.zhl19l1.mongodb.net
2 // Select the database to use.
3 use("streamflix")
4 db.conteudos.updateOne(
5   { titulo: "Matrix" },
6   { $inc: { visualizacoes: 1 } }
7 )
8
```

The right pane shows the result of the command:

```
1 {
2   acknowledged: true,
3   insertedId: null,
4   matchedCount: NumberInt('1'),
5   modifiedCount: NumberInt('1'),
6   upsertedCount: NumberInt('0')
7 }
```

Exercício 31

Incremente em 1000 a quantidade de visualizações de todos os conteúdos disponíveis.

```
R: db.conteudos.updateMany(
    { disponivel: true },
    { $inc: { visualizacoes: 1000 } }
)
```



```
eduard > OneDrive > Documentos > UEM > TERCEIRO > BDII > JS playground-1.mongodb.js >
  Connected to banco.zhl1911.mongodb.net
  // Select the database to use.
  use("streamflix")
  db.conteudos.updateMany(
    { disponivel: true },
    { $inc: { visualizacoes: 1000 } }
  )
  {
    acknowledged: true,
    insertedId: null,
    matchedCount: NumberInt('15'),
    modifiedCount: NumberInt('15'),
    upsertedCount: NumberInt('0')
  }
```

Exercício 32

Adicione o gênero Clássico ao filme Matrix.

```
R: db.conteudos.updateOne(
  { titulo: "Matrix" },
  { $push: { generos: "Clássico" } }
)
```

```
eduard > OneDrive > Documentos > UEM > TERCEIRO > BDII > JS playground-1.mongodb.js >
  Connected to banco.zhl1911.mongodb.net
  // Select the database to use.
  use("streamflix")
  db.conteudos.updateOne(
    { titulo: "Matrix" },
    { $push: { generos: "Clássico" } }
  )
  {
    acknowledged: true,
    insertedId: null,
    matchedCount: NumberInt('1'),
    modifiedCount: NumberInt('1'),
    upsertedCount: NumberInt('0')
  }
```

Para consultar: db.conteudos.find({ titulo: "Matrix" })

```
Connected to banco.zhl1911.mongodb.net
// Select the database to use.
use("streamflix")
db.conteudos.find({ titulo: "Matrix" })
{
  _id: ObjectId('6a3a898707a0ceaef28f2ff1'),
  titulo: 'Matrix',
  tipo: 'filme',
  ano: NumberInt('1999'),
  generos: [
    'Ação',
    'Ficção Científica',
    'Clássico'
  ],
  avaliacaoMedia: Double('9.2'),
  duracaoMinutos: NumberInt('136'),
  disponivel: true,
  visualizacoes: NumberInt('3101001'),
  diretor: { ... },
  elenco: [ ... ],
  idiomaOriginal: 'Inglês'
}
```

Exercício 33

Remova o gênero Clássico do filme Matrix.

```
R: db.conteudos.updateOne(
  { titulo: "Matrix" },
  { $pull: { generos: "Clássico" } }
```

```

)

MongoDB for VS Code
MongoDB playground-1.mongodb.js
Connected to banco.zhl1911.mongodb.net
// Select the database to use.
1 use("streamflix")
2 db.conteudos.updateOne(
3   { titulo: "Matrix" },
4   { $pull: { generos: "Clássico" } }
5 )
6
7
8
Playground Result
1 {
2   acknowledged: true,
3   insertedId: null,
4   matchedCount: NumberInt('1'),
5   modifiedCount: NumberInt('1'),
6   upsertedCount: NumberInt('0')
7 }

```

Para consultar: `db.conteudos.find({ titulo: "Matrix" })`

```

Connected to banco.zhl1911.mongodb.net
// Select the database to use.
use("streamflix")
db.conteudos.find({ titulo: "Matrix" })

Insert Document
Refresh

{
  _id: ObjectId('6a3a898707a0ceaf28f2ff1'),
  titulo: 'Matrix',
  tipo: 'filme',
  ano: NumberInt('1999'),
  generos: [
    'Ação',
    'Ficção Científica'
  ],
  avaliacaoMedia: Double('9.2'),
  duracaoMinutos: NumberInt('136'),
  disponivel: true,
  visualizacoes: NumberInt('3101001'),
  diretor: { ... },
  elenco: [ ... ],
  idiomaOriginal: 'Inglês'
}

```

Exercício 34

Remova o campo `telefone` da usuária Beatriz Nunes.

```

R: db.usuarios.updateOne(
    { nome: "Beatriz Nunes" },
    { $unset: { telefone: "" } }
)

```

```

MongoDB for VS Code
MongoDB playground-1.mongodb.js
Connected to banco.zhl1911.mongodb.net
// Select the database to use.
1 use("streamflix")
2 db.usuarios.updateOne(
3   { nome: "Beatriz Nunes" },
4   { $unset: { telefone: "" } }
5 )
6
7
8
Playground Result
1 {
2   acknowledged: true,
3   insertedId: null,
4   matchedCount: NumberInt('1'),
5   modifiedCount: NumberInt('1'),
6   upsertedCount: NumberInt('0')
7 }

```

Exercício 35

Adicione o benefício `sem anúncios` aos usuários do plano `Premium` na coleção `assinaturas`.

```

R: db.assinaturas.updateMany(

```

```

    { plano: "Premium" },
    { $push: { beneficios: "sem anúncios" } }
  )
}

```

The screenshot shows the MongoDB Playground interface. On the left, the command window displays the following JavaScript code:

```

// Connected to banco.zhl1911.mongodb.net
// Select the database to use.
use("streamflix")
db.assinaturas.updateMany(
  { plano: "Premium" },
  { $push: { beneficios: "sem anúncios" } }
)

```

On the right, the 'Playground Result' tab shows the execution result:

```

1 {
2   acknowledged: true,
3   insertedId: null,
4   matchedCount: NumberInt('2'),
5   modifiedCount: NumberInt('2'),
6   upsertedCount: NumberInt('0')
7 }

```

3.7. Operadores lógicos

Exercício 36

Liste os conteúdos que são filmes e possuem avaliação média maior que 9.

```

R: db.conteudos.find({
  $and: [
    { tipo: "filme" },
    { avaliacaoMedia: { $gt: 9 } }
  ]
})

```

The screenshot shows the MongoDB Playground interface. On the left, the command window displays the following JavaScript code:

```

// Connected to banco.zhl1911.mongodb.net
// Select the database to use.
use("streamflix")
db.conteudos.find({
  $and: [
    { tipo: "filme" },
    { avaliacaoMedia: { $gt: 9 } }
  ]
})

```

On the right, the 'Playground Result' tab shows the execution result, displaying two document objects:

```

{
  _id: ObjectId('6a3a898707a0ceaf28f2ff0'),
  titulo: 'Interestelar',
  tipo: 'filme',
  ano: NumberInt('2014'),
  generos: [...],
  avaliacaoMedia: Double('9.5'),
  duracaoMinutos: NumberInt('169'),
  disponivel: true,
  visualizacoes: NumberInt('2501000'),
  diretor: {...},
  elenco: [...],
  classificacao: '10+'
}

{
  _id: ObjectId('6a3a898707a0ceaf28f2ff1'),
  titulo: 'Matrix',
  tipo: 'filme',
  ano: NumberInt('1999'),
  generos: [...],
  avaliacaoMedia: Double('9.2'),
  duracaoMinutos: NumberInt('136'),
  disponivel: true,
  visualizacoes: NumberInt('3101001'),
  diretor: {...},
  elenco: [...],
  idiomaOriginal: 'Inglês'
}

```

```
Connected to bancozin191.mongodb.net
// Select the database to use.
use("streamflix")
db.conteudos.find({
  $and: [
    { tipo: "filme" },
    { avaliacaoMedia: { $gt: 9 } }
  ]
})
```

```
> diretor: { ...
},
> elenco: [ ...
],
idiomaOriginal: 'Inglês'
}

{
  _id: ObjectId('6a3a898707a0ceaf28f2ff3'),
  titulo: 'O Senhor dos Anéis: A Sociedade do Anel',
  tipo: 'filme',
  ano: NumberInt('2001'),
  generos: [ ...
],
  avaliacaoMedia: Double('9.4'),
  duracaoMinutos: NumberInt('178'),
  disponivel: true,
  visualizacoes: NumberInt('2201000'),
  diretor: { ...
},
  premios: [ ...
]
}

{
  _id: ObjectId('6a3a898707a0ceaf28f2ff4'),
  titulo: 'A Origem',
  tipo: 'filme',
  ano: NumberInt('2010'),
  generos: [ ...
],
  avaliacaoMedia: Double('9.1'),
  duracaoMinutos: NumberInt('148'),
  disponivel: true,
  visualizacoes: NumberInt('1951000'),
  diretor: { ...
}
}
```

Exercício 37

Liste os usuários que são de Curitiba ou de Maringá.

```
R: db.usuarios.find({
  $or: [
    { cidade: "Curitiba" },
    { cidade: "Maringá" }
  ]
})
```

```
Connected to banco.zhl19f1.mongodb.net
// Select the database to use.
use("streamflix")
db.usuarios.find({
  $or: [
    { cidade: "Curitiba" },
    { cidade: "Maringá" }
  ]
})
```

```
{
  "_id": ObjectId("6a3a8911551fbf74bf9873de"),
  "nome": "Ana Souza",
  "email": "ana@email.com",
  "idade": NumberInt("22"),
  "cidade": "Curitiba",
  "estado": "PR",
  "interesses": [ ... ],
  "ativo": true,
  "endereco": { ... }
}
```

```
{
  "_id": ObjectId("6a3a8911551fbf74bf9873df"),
  "nome": "Carlos Lima",
  "email": "carlos@email.com",
  "idade": NumberInt("31"),
  "cidade": "Maringá",
  "estado": "PR",
  "interesses": [ ... ],
  "ativo": true,
  "endereco": { ... }
}
```

```
{
  "_id": ObjectId("6a3a8911551fbf74bf9873e1"),
  "nome": "João Mendes",
  "email": "joao@email.com",
  "idade": NumberInt("19"),
  "cidade": "Curitiba",
  "estado": "PR",
  "interesses": [ ... ],
  "ativo": true
}
```

```
Connected to banco.zhl19f1.mongodb.net
1 // Select the database to use.
2 use("streamflix")
3 db.usuarios.find({
4   $or: [
5     { cidade: "Curitiba" },
6     { cidade: "Maringá" }
7   ]
8 })
9
```

```
+ Insert Document Refresh 10
```

```
{
  "_id": ObjectId("6a3a8911551fbf74bf9873e4"),
  "nome": "Beatriz Nunes",
  "email": "beatriz@email.com",
  "idade": NumberInt("24"),
  "cidade": "Maringá",
  "estado": "PR",
  "interesses": [ ... ],
  "ativo": true
}
```

```
{
  "_id": ObjectId("6a3a8911551fbf74bf9873e7"),
  "nome": "Eduardo Martins",
  "email": "eduardo@email.com",
  "idade": NumberInt("33"),
  "cidade": "Curitiba",
  "estado": "PR",
  "interesses": [ ... ],
  "ativo": true,
  "premium": false
}
```

```
{
  "_id": ObjectId("6a3a8e4abde8e249132c23bf"),
  "nome": "Eduarda Barragan",
  "email": "eduarda@email.com",
  "idade": NumberInt("19"),
  "cidade": "Maringá",
  "estado": "PR",
  "interesses": [ ... ],
  "ativo": true
}
```

Exercício 38

Liste os conteúdos que são séries ou documentários.

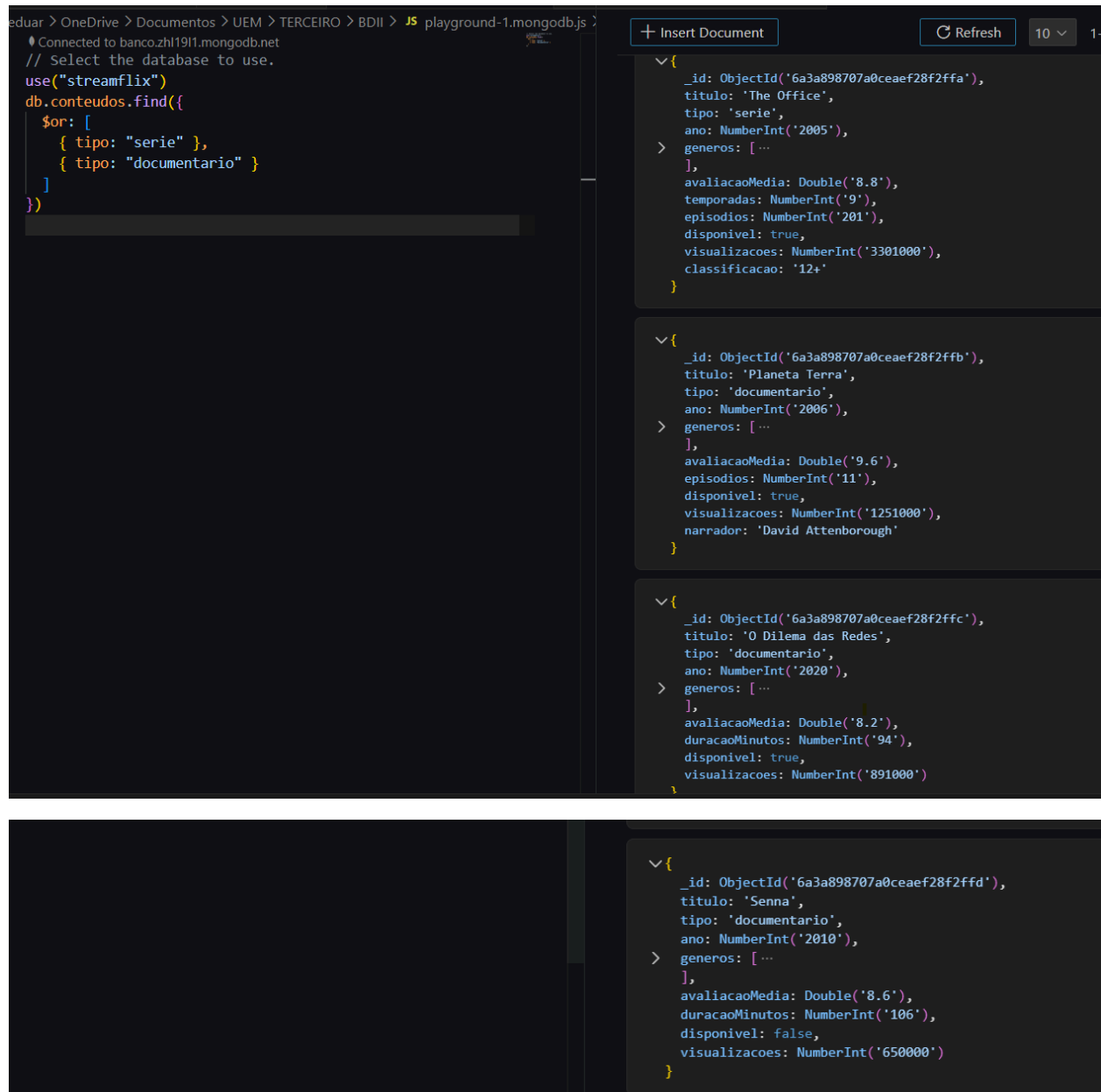
```
R: db.conteudos.find({
  $or: [
    { tipo: "serie" },
    { tipo: "documentario" }
  ]
})
```

The screenshot shows a MongoDB interface with a query on the left and results on the right. The query is:

```
use("streamflix")
db.conteudos.find({
  $or: [
    { tipo: "serie" },
    { tipo: "documentario" }
  ]
})
```

The results on the right show three documents:

- Document 1:**
 - _id: ObjectId('6a3a898707a0ceaf28f2ff7')
 - titulo: 'Dark'
 - tipo: 'serie'
 - ano: NumberInt('2017')
 - generos: [...]
 - avaliacaoMedia: Double('9.1')
 - temporadas: NumberInt('3')
 - episodios: NumberInt('26')
 - disponivel: true
 - visualizacoes: NumberInt('2101000')
 - classificacao: '16+'
- Document 2:**
 - _id: ObjectId('6a3a898707a0ceaf28f2ff8')
 - titulo: 'Breaking Bad'
 - tipo: 'serie'
 - ano: NumberInt('2008')
 - generos: [...]
 - avaliacaoMedia: Double('9.8')
 - temporadas: NumberInt('5')
 - episodios: NumberInt('62')
 - disponivel: true
 - visualizacoes: NumberInt('4201000')
 - classificacao: '18+'
- Document 3:**
 - _id: ObjectId('6a3a898707a0ceaf28f2ff9')
 - titulo: 'Stranger Things'
 - tipo: 'serie'
 - ano: NumberInt('2016')
 - generos: [...]
 - avaliacaoMedia: Double('8.9')



Exercício 39

Liste os conteúdos que possuem avaliação maior que 9 e visualizações acima de 2000000.

```
R: db.conteudos.find({
  $and: [
    { avaliacaoMedia: { $gt: 9 } },
    { visualizacoes: { $gt: 2000000 } }
  ]
})
```

```

Connected to banco.zhl1911.mongodb.net
// Select the database to use.
use("streamflix")
db.conteudos.find({
  $and: [
    { avaliacaoMedia: { $gt: 9 } },
    { visualizacoes: { $gt: 2000000 } }
  ]
})

```

```

{
  _id: ObjectId('6a3a898707a0ceaf28f2ff0'),
  titulo: 'Interstellar',
  tipo: 'filme',
  ano: NumberInt('2014'),
  generos: [ ... ],
  avaliacaoMedia: Double('9.5'),
  duracaoMinutos: NumberInt('169'),
  disponivel: true,
  visualizacoes: NumberInt('2501000'),
  diretor: { ... },
  elenco: [ ... ],
  classificacao: '10+'
}

```

```

{
  _id: ObjectId('6a3a898707a0ceaf28f2ff1'),
  titulo: 'Matrix',
  tipo: 'filme',
  ano: NumberInt('1999'),
  generos: [ ... ],
  avaliacaoMedia: Double('9.2'),
  duracaoMinutos: NumberInt('136'),
  disponivel: true,
  visualizacoes: NumberInt('3101001'),
  diretor: { ... },
  elenco: [ ... ],
  idiomaOriginal: 'Inglês'
}

```

```

Connected to banco.zhl1911.mongodb.net
// Select the database to use.
use("streamflix")
db.conteudos.find({
  $and: [
    { avaliacaoMedia: { $gt: 9 } },
    { visualizacoes: { $gt: 2000000 } }
  ]
})

```

+ Insert Document

Refresh

10 1-5

```

{
  _id: ObjectId('6a3a898707a0ceaf28f2ff3'),
  titulo: 'O Senhor dos Anéis: A Sociedade do Anel',
  tipo: 'filme',
  ano: NumberInt('2001'),
  generos: [ ... ],
  avaliacaoMedia: Double('9.4'),
  duracaoMinutos: NumberInt('178'),
  disponivel: true,
  visualizacoes: NumberInt('2201000'),
  diretor: { ... },
  premios: [ ... ]
}

```

```

{
  _id: ObjectId('6a3a898707a0ceaf28f2ff7'),
  titulo: 'Dark',
  tipo: 'serie',
  ano: NumberInt('2017'),
  generos: [ ... ],
  avaliacaoMedia: Double('9.1'),
  temporadas: NumberInt('3'),
  episodios: NumberInt('26'),
  disponivel: true,
  visualizacoes: NumberInt('2101000'),
  classificacao: '16+'
}

```

```

{
  _id: ObjectId('6a3a898707a0ceaf28f2ff8'),
  titulo: 'Breaking Bad',
  tipo: 'serie',
  ano: NumberInt('2008'),
  generos: [ ... ],
  avaliacaoMedia: Double('9.8'),
}

```


Exercício 40

Liste os usuários ativos com idade menor que 30.

```
R: db.usuarios.find({
  $and: [
    { ativo: true },
    { idade: { $lt: 30 } }
  ]
})
```

```
// Select the database to use.
use("streamflix")
db.usuarios.find({
  $and: [
    { ativo: true },
    { idade: { $lt: 30 } }
  ]
})
```

```
{
  "_id": ObjectId("6a3a8911551fbf74bf9873de"),
  "nome": "Ana Souza",
  "email": "ana@email.com",
  "idade": NumberInt("22"),
  "cidade": "Curitiba",
  "estado": "PR",
  "interesses": [ ... ],
  "ativo": true,
  "endereço": { ... }
}
```

```
{
  "_id": ObjectId("6a3a8911551fbf74bf9873e0"),
  "nome": "Fernanda Rocha",
  "email": "fernanda@email.com",
  "idade": NumberInt("27"),
  "cidade": "Londrina",
  "estado": "PR",
  "interesses": [ ... ],
  "ativo": true
}
```

```
{
  "_id": ObjectId("6a3a8911551fbf74bf9873e1"),
  "nome": "João Mendes",
  "email": "joao@email.com",
  "idade": NumberInt("19"),
  "cidade": "Curitiba",
  "estado": "PR",
  "interesses": [ ... ],
  "ativo": true
}
```

```
// Select the database to use.
use("streamflix")
db.usuarios.find({
  $and: [
    { ativo: true },
    { idade: { $lt: 30 } }
  ]
})
```

```
}
```

```

{
  _id: ObjectId('6a3a8911551fbf74bf9873e4'),
  nome: 'Beatriz Nunes',
  email: 'beatriz@email.com',
  idade: NumberInt('24'),
  cidade: 'Maringá',
  estado: 'PR',
  interesses: [ ... ],
  ativo: true
}
```

```

{
  _id: ObjectId('6a3a8911551fbf74bf9873e5'),
  nome: 'Lucas Ferreira',
  email: 'lucas@email.com',
  idade: NumberInt('29'),
  cidade: 'Florianópolis',
  estado: 'SC',
  interesses: [ ... ],
  ativo: true
}
```

```

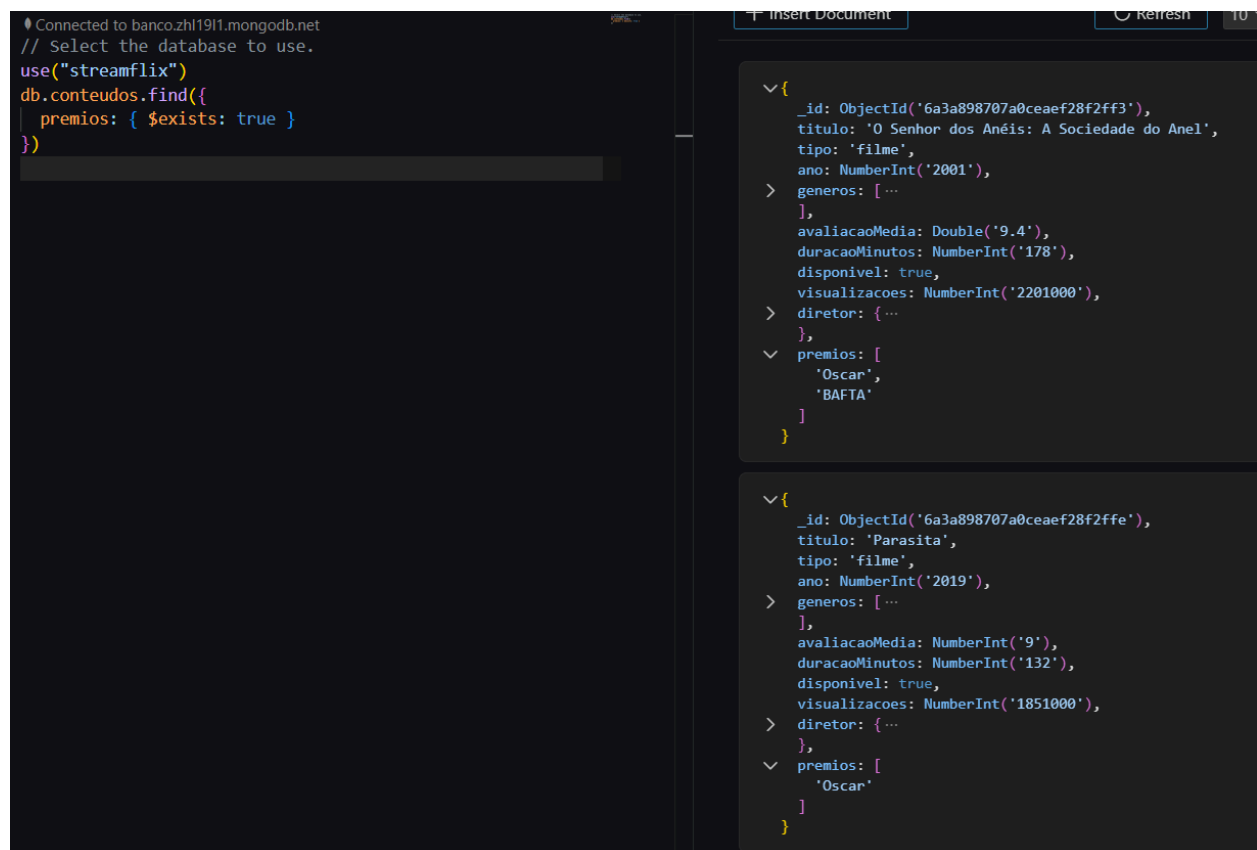
{
  _id: ObjectId('6a3a8e4abde8e249132c23bf'),
  nome: 'Eduarda Barragan',
  email: 'eduarda@email.com',
  idade: NumberInt('19'),
  cidade: 'Maringá',
  estado: 'PR',
  interesses: [ ... ],
  ativo: true
}
```

3.8. Campos opcionais e flexibilidade NoSQL

Exercício 41

Liste os conteúdos que possuem o campo `premios`.

```
R: db.conteudos.find({
  premios: { $exists: true }
})
```



Exercício 42

Liste os conteúdos que não possuem o campo diretor.

```
R: db.conteudos.find({
  diretor: { $exists: false }
})
```

Connected to banco.zhl19l1.mongodb.net
// Select the database to use.
use("streamflix")
db.conteudos.find({
 diretor: { \$exists: false }
})

+ Insert Document Refresh 1

```
{  
  "_id": ObjectId('6a3a898707a0ceaef28f2ff5'),  
  "titulo": "Divertida Mente",  
  "tipo": "filme",  
  "ano": NumberInt('2015'),  
  "generos": [...],  
  "avaliacaoMedia": Double('8.7'),  
  "duracaoMinutos": NumberInt('95'),  
  "disponivel": true,  
  "visualizacoes": NumberInt('1751000')  
}
```

```
{  
  "_id": ObjectId('6a3a898707a0ceaef28f2ff7'),  
  "titulo": "Dark",  
  "tipo": "serie",  
  "ano": NumberInt('2017'),  
  "generos": [...],  
  "avaliacaoMedia": Double('9.1'),  
  "temporadas": NumberInt('3'),  
  "episodios": NumberInt('26'),  
  "disponivel": true,  
  "visualizacoes": NumberInt('2101000'),  
  "classificacao": "16+"  
}
```

equal / OneDrive / Documentos / GEM / TERCEIRO / BDII / playground-1.mongodb.js
Connected to banco.zhl19l1.mongodb.net
// Select the database to use.
use("streamflix")
db.conteudos.find({
 diretor: { \$exists: false }
})

+ Insert Document Refresh 10

```
}
```

```
{  
  "_id": ObjectId('6a3a898707a0ceaef28f2ff8'),  
  "titulo": "Breaking Bad",  
  "tipo": "serie",  
  "ano": NumberInt('2008'),  
  "generos": [...],  
  "avaliacaoMedia": Double('9.8'),  
  "temporadas": NumberInt('5'),  
  "episodios": NumberInt('62'),  
  "disponivel": true,  
  "visualizacoes": NumberInt('4201000'),  
  "classificacao": "18+"  
}
```

```
{  
  "_id": ObjectId('6a3a898707a0ceaef28f2ff9'),  
  "titulo": "Stranger Things",  
  "tipo": "serie",  
  "ano": NumberInt('2016'),  
  "generos": [...],  
  "avaliacaoMedia": Double('8.9'),  
  "temporadas": NumberInt('4'),  
  "episodios": NumberInt('34'),  
  "disponivel": true,  
  "visualizacoes": NumberInt('3901000'),  
  "classificacao": "14+"  
}
```

eduar > OneDrive > Documentos > UEM > TERCEIRO > BDII > JS playground-1.mongodb.js

```
Connected to banco.zhl19l1.mongodb.net
// Select the database to use.
use("streamflix")
db.conteudos.find({
  diretor: { $exists: false }
})
```

+ Insert Document Refresh 10

```
{
  _id: ObjectId('6a3a898707a0ceaf28f2ffa'),
  titulo: 'The Office',
  tipo: 'serie',
  ano: NumberInt('2005'),
  generos: [ ... ],
  avaliacaoMedia: Double('8.8'),
  temporadas: NumberInt('9'),
  episodios: NumberInt('201'),
  disponivel: true,
  visualizacoes: NumberInt('3301000'),
  classificacao: '12+'
}
```

```
{
  _id: ObjectId('6a3a898707a0ceaf28f2ffb'),
  titulo: 'Planeta Terra',
  tipo: 'documentario',
  ano: NumberInt('2006'),
  generos: [ ... ],
  avaliacaoMedia: Double('9.6'),
  episodios: NumberInt('11'),
  disponivel: true,
  visualizacoes: NumberInt('1251000'),
  narrador: 'David Attenborough'
}
```

eduar > OneDrive > Documentos > UEM > TERCEIRO > BDII > JS playground-1.mongodb.js

```
Connected to banco.zhl19l1.mongodb.net
// Select the database to use.
use("streamflix")
db.conteudos.find({
  diretor: { $exists: false }
})
```

+ Insert Document Refresh

```
{
  _id: ObjectId('6a3a898707a0ceaf28f2ffc'),
  titulo: 'O Dilema das Redes',
  tipo: 'documentario',
  ano: NumberInt('2020'),
  generos: [ ... ],
  avaliacaoMedia: Double('8.2'),
  duracaoMinutos: NumberInt('94'),
  disponivel: true,
  visualizacoes: NumberInt('891000')
}
```

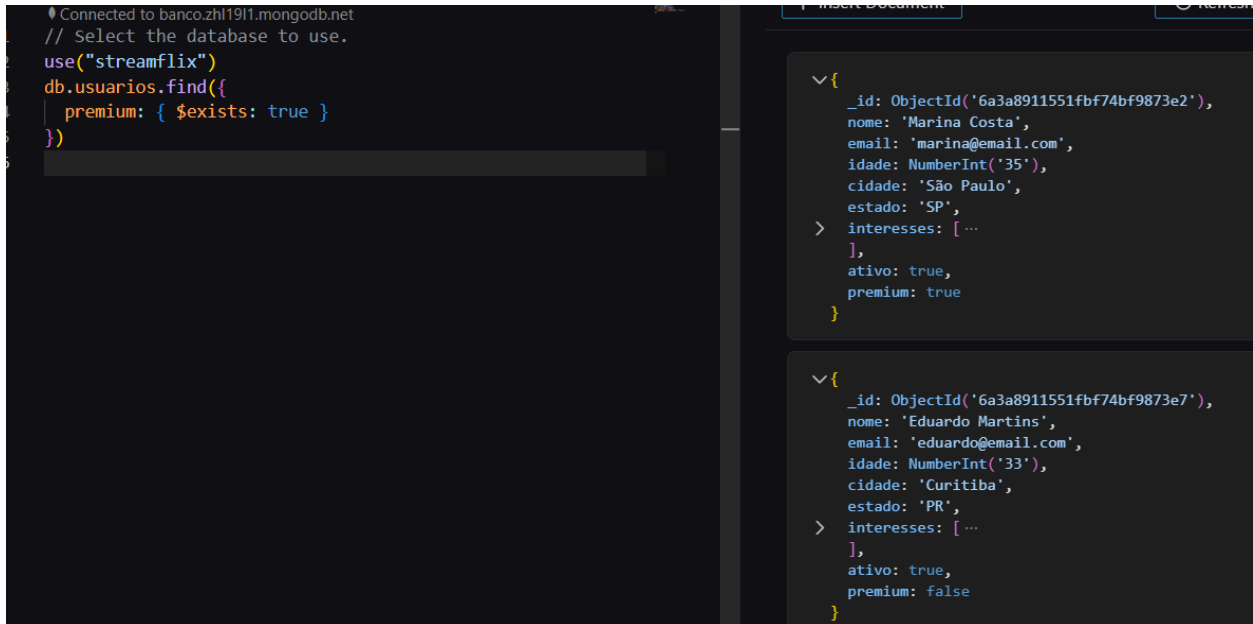
```
{
  _id: ObjectId('6a3a898707a0ceaf28f2ffd'),
  titulo: 'Senna',
  tipo: 'documentario',
  ano: NumberInt('2010'),
  generos: [ ... ],
  avaliacaoMedia: Double('8.6'),
  duracaoMinutos: NumberInt('106'),
  disponivel: false,
  visualizacoes: NumberInt('650000')
}
```

```
{
  _id: ObjectId('6a3a8e880ccf93277cd2811f'),
  titulo: 'Como Eu Era Antes de Você',
  tipo: 'filme',
  ano: NumberInt('2016'),
  generos: [ ... ],
  avaliacaoMedia: NumberInt('8'),
  duracaoMinutos: NumberInt('110'),
  disponivel: true,
  visualizacoes: NumberInt('1000')
}
```

Exercício 43

Liste os usuários que possuem o campo `premium`.

```
R: db.usuarios.find({  
    premium: { $exists: true }  
})
```



The screenshot shows a MongoDB CLI session. On the left, the command prompt shows the user is connected to 'banco.zhi19f1.mongodb.net' and has selected the 'streamflix' database. The query `db.usuarios.find({ premium: { $exists: true } })` has been executed. On the right, the results are displayed as two JSON documents. The first document is for a user named 'Marina Costa' with a premium status of true. The second document is for a user named 'Eduardo Martins' with a premium status of false.

```
Connected to banco.zhi19f1.mongodb.net
// Select the database to use.
use("streamflix")
db.usuarios.find({
  premium: { $exists: true }
})
```

```
{
  "_id": ObjectId("6a3a8911551fbf74bf9873e2"),
  "nome": "Marina Costa",
  "email": "marina@email.com",
  "idade": NumberInt("35"),
  "cidade": "São Paulo",
  "estado": "SP",
  "interesses": [ ... ],
  "ativo": true,
  "premium": true
}
```

```
{
  "_id": ObjectId("6a3a8911551fbf74bf9873e7"),
  "nome": "Eduardo Martins",
  "email": "eduardo@email.com",
  "idade": NumberInt("33"),
  "cidade": "Curitiba",
  "estado": "PR",
  "interesses": [ ... ],
  "ativo": true,
  "premium": false
}
```

Exercício 44

Liste os conteúdos que possuem o campo `temporadas`.

```
R: db.conteudos.find({  
    temporadas: { $exists: true }  
})
```

```
connected to banco.zm1911.mongodb.net
// Select the database to use.
use("streamflix")
db.conteudos.find({
  temporadas: { $exists: true }
})
```

```
{
  "_id": ObjectId('6a3a898707a0ceaef28f2ff7'),
  "titulo": "Dark",
  "tipo": "serie",
  "ano": NumberInt('2017'),
  "generos": [ ... ],
  "avaliacaoMedia": Double('9.1'),
  "temporadas": NumberInt('3'),
  "episodios": NumberInt('26'),
  "disponivel": true,
  "visualizacoes": NumberInt('2101000'),
  "classificacao": "16+"
}
```

```
{
  "_id": ObjectId('6a3a898707a0ceaef28f2ff8'),
  "titulo": "Breaking Bad",
  "tipo": "serie",
  "ano": NumberInt('2008'),
  "generos": [ ... ],
  "avaliacaoMedia": Double('9.8'),
  "temporadas": NumberInt('5'),
  "episodios": NumberInt('62'),
  "disponivel": true,
  "visualizacoes": NumberInt('4201000'),
  "classificacao": "18+"
}
```

```
{
  "_id": ObjectId('6a3a898707a0ceaef28f2ff9'),
  "titulo": "Stranger Things",
  "tipo": "serie",
  "ano": NumberInt('2016'),
  "generos": [ ... ],
  "avaliacaoMedia": Double('8.9'),
  "temporadas": NumberInt('4'),
  "episodios": NumberInt('34'),
  "disponivel": true,
  "visualizacoes": NumberInt('3901000'),
  "classificacao": "14+"
}
```

```
{
  "_id": ObjectId('6a3a898707a0ceaef28f2ffa'),
  "titulo": "The Office",
  "tipo": "serie",
  "ano": NumberInt('2005'),
  "generos": [ ... ],
  "avaliacaoMedia": Double('8.8'),
  "temporadas": NumberInt('9'),
  "episodios": NumberInt('201'),
  "disponivel": true,
  "visualizacoes": NumberInt('3301000'),
  "classificacao": "12+"
}
```

Exercício 45

Explique por que os documentos da coleção `conteudos` podem ter campos diferentes.

R: Os documentos da coleção `conteudos` podem ter campos diferentes porque ela armazena tipos distintos de mídia, como filmes, séries e documentários. Essa possibilidade é uma característica fundamental em bancos de dados NoSQL, permitindo que documentos com estruturas semelhantes sejam armazenados em uma mesma coleção.

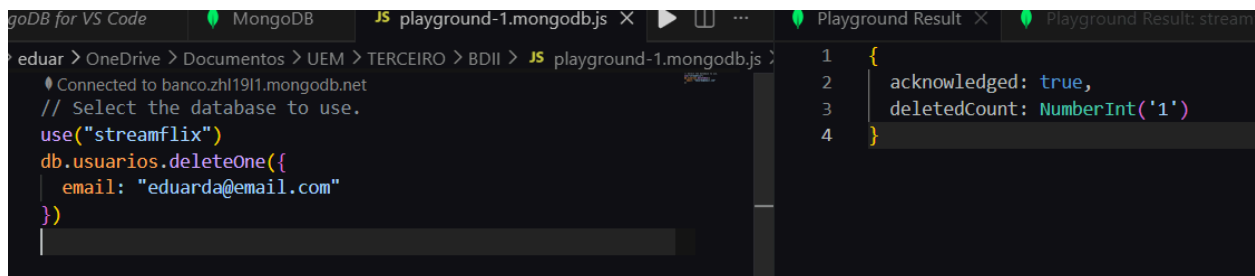
Dessa forma, é possível adaptar a estrutura do documento para acomodar os atributos específicos de cada tipo de conteúdo (por exemplo, um filme pode ter um campo de duração, enquanto uma série pode ter um campo de número de temporadas).

3.9. Remoção de documentos

Exercício 46

Remova o usuário que você criou no Exercício 6.

```
db.usuarios.deleteOne({  
  email: "eduarda@email.com"  
})
```

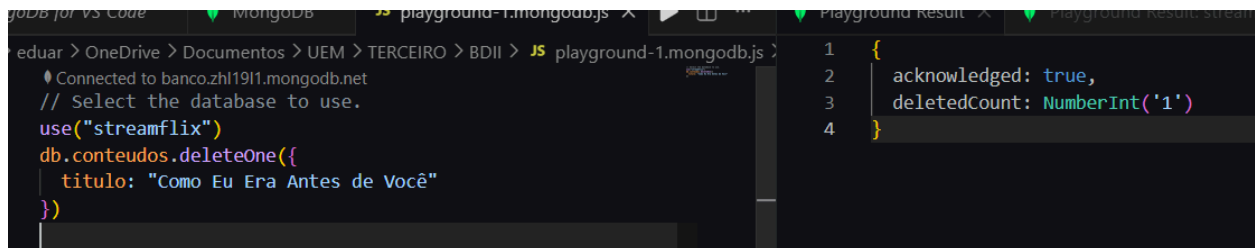


The screenshot shows the MongoDB Playground interface. The left pane displays the command: `use("streamflix"); db.usuarios.deleteOne({ email: "eduarda@email.com" });`. The right pane shows the result: `{ acknowledged: true, deletedCount: NumberInt('1') }`.

Exercício 47

Remova o conteúdo que você criou no Exercício 7.

```
db.conteudos.deleteOne({  
  titulo: "Como Eu Era Antes de Você"  
})
```

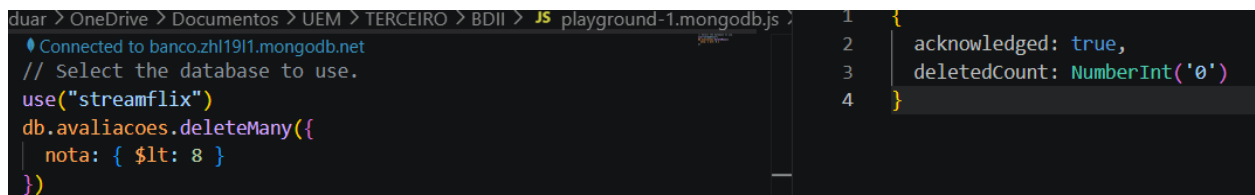


The screenshot shows the MongoDB Playground interface. The left pane displays the command: `use("streamflix"); db.conteudos.deleteOne({ titulo: "Como Eu Era Antes de Você" });`. The right pane shows the result: `{ acknowledged: true, deletedCount: NumberInt('1') }`.

Exercício 48

Remova todas as avaliações com nota menor que 8.

```
db.avaliacoes.deleteMany({  
  nota: { $lt: 8 }  
})
```



The screenshot shows the MongoDB Playground interface. The left pane displays the command: `use("streamflix"); db.avaliacoes.deleteMany({ nota: { $lt: 8 } });`. The right pane shows the result: `{ acknowledged: true, deletedCount: NumberInt('0') }`.

Exercício 49

Remova os registros de histórico cujo progresso seja menor que 40.


```
db.historico.deleteMany({
  progressoPercentual: { $lt: 40 }
})
```

```
> eduar > OneDrive > Documentos > UEM > TERCEIRO > BDII > JS playground-1.mongodb.js > 1 {
  2   acknowledged: true,
  3   deletedCount: NumberInt('1')
  4 }
```

```
Connected to banco.zhl19l1.mongodb.net
// Select the database to use.
use("streamflix")
db.historico.deleteMany({
  progressoPercentual: { $lt: 40 }
})
```

Exercício 50

Explique a diferença entre manter as informações separadas em várias coleções e armazenar tudo em um único documento.

R: A principal diferença entre manter informações separadas em várias coleções e armazenar tudo em um único documento está na relação entre desempenho e flexibilidade de dados.

Ao armazenar tudo em um documento, é possível centralizar todas as informações relacionadas em um único lugar, favorecendo o desempenho de leitura e permitindo recuperar o registro completo em uma única consulta. Porém, essa abordagem pode resultar em documentos excessivamente grandes e tornar atualizações complexas.

Por outro lado, separar as informações em várias coleções utilizando referências oferece maior flexibilidade e integridade, permitindo que os dados cresçam sem limites e facilitando atualizações frequentes, embora o custo computacional seja maior devido à necessidade de consultas adicionais ou operações de junção.

Exercício 51

Explique uma vantagem e uma desvantagem de usar documentos aninhados no MongoDB.

R: Uma das vantagens de usar documentos aninhados é o desempenho de leitura, uma vez que centralizar todas as informações relacionadas em um único documento permite recuperar o registro completo e todos os seus detalhes em uma única consulta, sem a necessidade de operações adicionais.

Uma desvantagem é que essa abordagem pode resultar em documentos excessivamente grandes, fazendo com que o documento ultrapasse o limite de tamanho do MongoDB e torne as atualizações dos dados mais complexas conforme a estrutura cresce.

Exercício 52

Explique em quais situações seria melhor usar referência entre coleções.

R: O uso de referências entre coleções é ideal em situações onde os dados possuem relações de "um para muitos" ou "muitos para muitos" que tendem a crescer de forma ilimitada, evitando que o documento exceda o limite de tamanho do MongoDB.

Essa abordagem também é melhor quando os dados sofrem atualizações frequentes, uma vez que facilita a manutenção, e quando há necessidade de compartilhar informações entre diferentes entidades ou realizar consultas independentes sobre esses dados, garantindo maior flexibilidade e integridade às informações.

Exercício 53

Explique em quais situações seria melhor usar dados incorporados no mesmo documento.

R: O uso de dados incorporados no mesmo documento é recomendado especialmente para relacionamentos do tipo "um para poucos", onde a quantidade de informações relacionadas é limitada e não cresce indefinidamente.

Essa modelagem é ideal quando a prioridade é o desempenho de leitura, permitindo obter o registro completo em uma única consulta sem a necessidade de junções complexas.